

Elastic Multi-Gradient Descent for Parallel Continual Learning

Fan Lyu, *Member, IEEE*, Wei Feng, *Member, IEEE*, Yuepan Li, Qing Sun, Fanhua Shang, *Member, IEEE*, Liang Wan, *Member, IEEE*, Liang Wang, *Fellow, IEEE*

Abstract—Learning a single shared model across multiple tasks is a long-standing goal in machine learning, traditionally addressed through Multi-Task Learning (MTL) or Serial Continual Learning (SCL). However, MTL assumes fixed task availability, limiting adaptability to new tasks, while SCL suffers from delayed response due to its strictly sequential learning protocol. This paper studies the novel paradigm of Parallel Continual Learning (PCL) in dynamic multi-task scenarios, where a diverse set of tasks is encountered at different time points. PCL inherits the challenges of both MTL and SCL, namely, task conflict and catastrophic forgetting, compounded by the dynamic and asynchronous arrival of tasks. To address these challenges, we propose Elastic Multi-Gradient Descent (EMGD), which formulates PCL as a dynamic multi-objective optimization problem. EMGD introduces task-specific elastic factors to guide gradient updates toward Pareto-optimal directions, ensuring balanced learning across tasks. Additionally, we develop a gradient-guided memory editing mechanism that aligns rehearsal data with the optimized descent direction, mitigating memory-induced interference. Theoretical analysis shows that accumulation points of the EMGD update rule are Pareto critical under the proposed formulation, and extensive experiments on image classification benchmarks demonstrate that EMGD significantly outperforms existing methods, including state-of-the-art PCL, MTL, and SCL approaches.

Index Terms—continual learning, catastrophic forgetting, training conflict, multi-objective optimization, Pareto optimum.

I. INTRODUCTION

LEARNING a single shared model for multiple tasks has long been a prominent research topic [52], [36], with broad applications in domains such as recommendation systems [81] and machine translation [77]. Two primary paradigms have emerged for building such unified models: *Multi-Task Learning* (MTL) and *Continual Learning* (CL). These approaches mainly differ in their strategies for data accessibility and usage. An overview of MTL and CL is illustrated in Fig. 1(a) and (b).

MTL [72] assumes access to a fixed set of data streams corresponding to different tasks, which are trained simultaneously in parallel. A key challenge in MTL is task conflict arising from inter-task interference. To address this, traditional

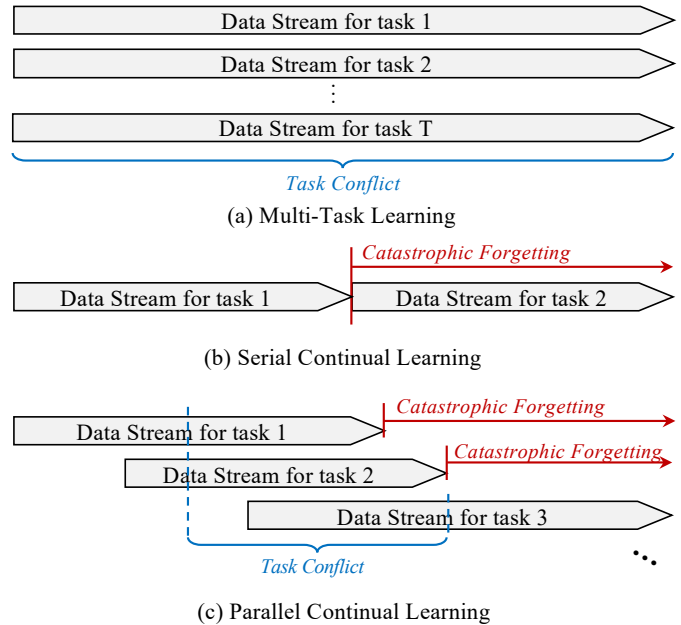


Fig. 1: Comparisons of Multi-Task Learning (MTL), Serial Continual Learning (SCL) and Parallel Continual Learning (PCL). (a) MTL relies on a fixed number of different tasks without adaptive incremental learning for new tasks. (b) SCL learns from sequential multiple tasks, where new tasks have to wait for the completion of previous training. (c) PCL enables training of multiple tasks based on their access time, allowing new tasks to be incorporated immediately.

MTL methods are broadly categorized into feature-based [54], [68] and parameter-based [66], [7] approaches. More recently, task balancing through reweighting has attracted increasing attention and can be further grouped into learning-based [18], [44], solving-based [61], and calculating-based [45], [75], [41] methods. Different from MTL, CL [33], [37], [49] aims to continuously learn new knowledge from a sequence of tasks with non-overlapping data streams over a lifelong time. Because the data of each task comes in sequence, the challenge of CL is catastrophic forgetting of old tasks or finished tasks. The existing CL methods are grouped into three types [21], *i.e.*, regularization-based methods [33], [14], architecture-based methods [53], [73], and rehearsal-based methods [49], [16], [29], [51]. Specifically, rehearsal has been proven effective against forgetting, which stores a small sample from old tasks and retrains them together with new tasks. Although the data access and learning settings are different, both MTL and CL aim to train a single shared model capable of handling multiple

F. Lyu is with the Computer Vision Center (CVC), Universitat Autònoma de Barcelona (UAB), Barcelona, 08193 Spain. Contact: fanlyu@cvc.uab.cat.

W. Feng, Y. Li, F. Shang, L. Wan are with the College of Intelligence and Computing, Tianjin University, Tianjin, 300350 China. Contacts: ({yuepanli, fhshang, lwan}@tju.edu.cn, wfeng@ieee.org)

Q. Sun is with Tencent Inc., Beijing, 100190 China. Contact: sssun-qing@tju.edu.cn

L. Wang is with the New Laboratory of Pattern Recognition (NLPR), Institute of Automation, Chinese Academy of Sciences (CASIA), Beijing, 100190 China. Contacts: wangliang@nlpr.ia.ac.cn

The corresponding author is Wei Feng.

Manuscript received **, **, 20**, revised August **, **.

tasks. This model consists of a shared backbone that remains consistent across all tasks, along with task-specific heads that are dynamically added or updated as needed. Throughout the learning process, all tasks utilize the same backbone, ensuring parameter sharing and knowledge transfer.

Despite the success of MTL and traditional CL in learning a unified model for multiple tasks, they are still insufficient for learning in real-world multi-task changing environments. On the one hand, classical MTL assumes access to a fixed set of tasks and does not accommodate the sequential or incremental arrival of new tasks or environments, resulting in limited adaptability to novel knowledge. On the other hand, traditional CL, hereafter referred to as *Serial Continual Learning* (SCL), typically enforces a sequential task learning protocol, in which a new task can only be trained after the current task has been completed. This leads to delayed responsiveness to emerging task demands. To address these limitations, our previous work [50] introduced a more general and practical framework termed **Parallel Continual Learning** (PCL). Unlike SCL, which adheres to a one-pipe, strictly sequential data stream, PCL allows new tasks to be accessed and trained at any time, concurrently with ongoing tasks. In doing so, PCL bridges the gap between MTL and SCL by supporting both continual adaptation and immediate response to new task demands, as illustrated in Fig. 1(c). This flexible learning paradigm satisfies the core requirements of lifelong learning in dynamic multi-source environments. However, PCL inherits major challenges from both MTL and SCL, namely, *task conflict* and *catastrophic forgetting*. Due to concurrent training on diverse tasks, gradient interference can result in task conflict [61], where backpropagation fails to optimize all tasks simultaneously toward their respective objectives. Meanwhile, catastrophic forgetting [26], [33] persists due to the unavailability of data from previously completed tasks [5]. Importantly, in the context of PCL, these issues are further exacerbated by the uncertainty and irregularity in the arrival times of new tasks, making them more complex than in conventional MTL or SCL settings.

To address the challenges inherent in PCL, our previous work [50] introduced Maximum Discrepancy Optimization (MaxDO). MaxDO focuses on quantifying the discrepancy among gradients from multiple tasks and proposes an explicit metric asymmetric gradient distance for learning under gradient conflict in PCL. This metric accounts for both the magnitude ratio and directional alignment of task gradients, while introducing a tolerance mechanism that allows for flexibility in handling smaller gradients. Furthermore, MaxDO incorporates an optimization strategy that explicitly minimizes gradient discrepancy to mitigate self-interference during training. However, MaxDO primarily aims to balance multiple tasks heuristically and does not explicitly address the issue of negative transfer in gradient updates. As a result, it fails to consistently align updates with the Pareto-optimal direction, particularly in the dynamic and asynchronous setting of PCL.

Building on this foundation, this work further investigates the problem of PCL by simultaneously tackling the dual challenges of catastrophic forgetting and task conflict. We frame PCL as a dynamic multi-objective optimization (MOO) problem, with the objective of constraining updates to remain within

the Pareto front. Traditional MOO approaches, such as Multi-Gradient Descent Algorithm (MGDA) [23], may fail under task imbalance in PCL, leading to trivial updates that stagnate training, i.e., avoiding forgetting by abandoning model update. Instead, we propose a simple yet effective method called **Elastic Multi-Gradient Descent** (EMGD). EMGD extends the classical steepest descent approach [25] by introducing an elastic factor for each task. This factor serves to regulate the alignment between each task's gradient and the under-optimized gradient, based on both gradient magnitude variation and inter-gradient similarity. Specifically, gradients with larger magnitudes are subject to stricter constraints, while those with smaller magnitudes are allowed greater flexibility. The resulting optimization problem can be efficiently solved in its dual form via quadratic programming. We also provide theoretical analysis showing that accumulation points of the EMGD update rule are Pareto critical under the proposed formulation. In addition, we introduce a memory editing mechanism guided by the Pareto descent direction obtained from EMGD. By modifying the stored data accordingly, this mechanism helps reduce potential conflicts between old and new tasks and enhances the utility of replayed samples in subsequent training iterations.

Compared to MaxDO [50], EMGD formulates PCL as a dynamic MOO problem and explicitly seeks a non-trivial Pareto descent direction, which ensures that updates improve or at least do not degrade any active task. To achieve this, EMGD introduces task-specific elastic factors, allowing the optimizer to adaptively balance learning between new and old tasks. We provide three advantages of our new proposed EMGD against our previous work MaxDO [50]: (1) *Non-trivial Pareto descent direction*: EMGD guarantees a descent direction that does not worsen any task, and even improves all tasks without sacrificing any of them. (2) *Gradient-guided memory rehearsal*: EMGD uses the computed descent direction to guide memory editing, thereby reducing task conflict in rehearsal and improving retention. (3) *Improved empirical performance*: EMGD consistently outperforms MaxDO and other baselines in extensive experiments across PCL benchmarks, achieving new state-of-the-art results. In summary, while both MaxDO and EMGD address training conflict and forgetting in PCL, EMGD introduces a more theoretically grounded and practically effective framework by leveraging Pareto descent directions and memory editing. We conduct experiments on three PCL benchmark datasets, and the results show that the proposed EMGD clearly outperforms MaxDO, as well as state-of-the-art MTL and SCL methods. Our contributions are three-fold:

- (1) We introduce EMGD, a novel method for resolving challenges in PCL by formulating it as a dynamic MOO problem. EMGD incorporates task-specific elastic factors to balance training between new and previously learned tasks, ensuring both task adaptability and knowledge retention.
- (2) We propose a memory editing mechanism guided by the target gradient direction obtained from EMGD. This technique performs pixel-level adjustments to stored samples, effectively mitigating task interference and reducing catastrophic forgetting in the replay buffer.
- (3) We validate the effectiveness of the proposed method through extensive experiments. The results demonstrate that

EMGD outperforms SOTA approaches in PCL, MTL and SCL. EMGD achieves superior performance across diverse PCL settings, effectively addressing both task conflict and catastrophic forgetting.

II. RELATED WORK

A. Multi-Task Learning

Multi-Task Learning (MTL) [72] aims to learn multiple tasks jointly by exploiting shared knowledge across tasks, either within a single domain [42], [61], [47] or across domains [65]. Its main motivation is to improve generalization and learning efficiency through parameter sharing and task interaction [80]. However, joint optimization across tasks often introduces negative transfer and task conflict [79], where improving one task may hurt others. Existing MTL methods can be broadly grouped into three categories [80]. Feature-based methods learn shared task representations to capture common structure across tasks [40], [64], [3], [46], [43], [35]. Parameter-based methods model task relationships through shared, structured, or decomposed parameters [2], [78], [66], [74], [10], [13]. Weighting-based methods rebalance task losses or gradients during training, including learned weighting, Pareto-based optimization, and gradient projection strategies [18], [44], [61], [75], [27], [41]. Despite their success, conventional MTL typically assumes a fixed set of tasks and simultaneous access to all task data. These assumptions are restrictive in dynamic environments where tasks arrive progressively and data streams are only partially available. In such cases, standard joint training is no longer directly applicable, which motivates more flexible paradigms for evolving multi-task scenarios.

B. Continual Learning

Continual Learning (CL), also known as incremental learning, aims to acquire new knowledge from sequential data streams while preserving previously learned information [26]. Its core challenge is catastrophic forgetting, where adapting to new tasks may overwrite old knowledge. Depending on the evaluation protocol, CL can be studied under task-incremental or class-incremental settings, both of which are considered in this paper. Traditional CL typically follows a strictly sequential learning protocol, *i.e.*, task-by-task, which we refer to as Serial Continual Learning (SCL). Existing CL methods are commonly grouped into three categories [21]. Rehearsal-based methods alleviate forgetting by replaying stored samples, features, or generated data from previous tasks [49], [16], [17], [58], [15], [70], [48]. Regularization-based methods preserve prior knowledge by constraining parameter updates according to their estimated importance [33], [76], [1]. Architecture-based methods mitigate interference by expanding the model or routing different tasks through different parameter subsets [53], [60], [24], [63]. A related setting, sometimes referred to as multi-task continual learning, also considers learning multiple tasks over time. However, most existing methods in this direction still follow the classical SCL protocol, where tasks are optimized one at a time in a predefined order [55], [39]. Therefore, conventional CL remains limited in dynamic scenarios where new task streams may arrive before previous training has finished. In contrast,

this paper studies Parallel Continual Learning (PCL), where a new task can be accessed and trained at any time.

III. PARALLEL CONTINUAL LEARNING

A. Problem Definition: Parallel Continual Learning

Suppose we have a sequence of T tasks corresponding to T parallel data streams $\{\mathcal{D}_1, \dots, \mathcal{D}_T\}$, where each data stream can be accessed and suspended at any time. Each data stream consists of a sequence of data tuples $(\mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ is an input data point and $\mathbf{y}$ is a one-hot label vector. The number of classes for task t is denoted as C_t . For simplicity, we assume that each data stream is stationary, meaning that its data are sampled from a fixed but unknown probability distribution. We denote the start and end times of the t -th data stream as (s_t, e_t) . To simplify the temporal structure, we assume $s_{t-1} \leq s_t \leq 1 + \max(e_1, e_2, \dots, e_{t-1})$ for any task $t > 1$, implying that tasks are accessed sequentially from 1 to T and that there is no idle period on the timeline during which no data stream is active. It is worth noting that traditional SCL is a special case of PCL, where all tasks are processed strictly in sequence, *i.e.*, $\forall t > 1, s_t = e_{t-1} + 1$.

A PCL model consists of a *task-agnostic backbone* parameterized by θ , and a dynamically growing set of *task-specific classifiers* parameterized by θ_t . When a new task is accessed, a corresponding task-specific classifier is instantiated. The overall model is composed of two functions: $f_\theta : \mathbf{x} \mapsto \mathbf{v} \in \mathbb{R}^{1 \times D}$ and $f_{\theta_t} : \mathbf{v} \in \mathbb{R}^{1 \times D} \mapsto \mathbf{p} \in \mathbb{R}^{1 \times C_t}$, where $\mathbf{x}$ is the input sample, $\mathbf{v}$ is the task-agnostic feature representation, and $\mathbf{p}$ is the predicted class probability. Based on the above concepts, PCL is defined as follows.

Definition 1 (Parallel Continual Learning). *Parallel Continual Learning (PCL) describes a learning paradigm where a model is continuously trained on multiple tasks, each associated with a distinct temporal access period. PCL allows for a dynamic and potentially overlapping set of active tasks, enabling the model to adaptively learn from all available data at any given time.*

B. Dynamic Multi-Task Learning with Rehearsal

To mitigate forgetting of tasks whose data streams have concluded, we adopt the rehearsal strategy [49], [16], [29], [4], which stores a small subset of training data and introduces an additional memory stream for retraining previously completed tasks. Specifically, a fraction of data from each completed task is retained in a fixed memory buffer $\mathcal{D}_0$. At time t , the PCL model is trained using both the currently active data streams and the memory stream $\mathcal{D}_0$, leading to the following multi-objective empirical risk minimization problem:

$$\min_{\theta, \{\theta_i | i \in \mathcal{T}_t\}} \{\ell_i(\mathcal{D}_i) | i \in \mathcal{T}_t\}, \quad (1)$$

where γ and γ_i are the learning rates for the shared backbone and task-specific classifiers, respectively. Since each classifier θ_i is updated independently based on its own loss, we primarily focus on the update of θ . Let the negative gradient of task i with respect to θ be denoted by $\mathbf{g}_i = -\nabla_\theta \ell_i$. However, simply averaging gradients across tasks, as done in AvgGrad, treats all tasks equally and fails to guarantee Pareto optimality [12].

A straightforward solution to this dynamic MOO problem is to apply the Average Gradient (AvgGrad) method, in which the model parameters are updated as follows:

$$\theta \leftarrow \theta - \gamma \cdot \frac{1}{|\mathcal{T}_t|} \sum_{i \in \mathcal{T}_t} \nabla_{\theta} \ell_i, \quad (2)$$

$$\theta_i \leftarrow \theta_i - \gamma_i \cdot \nabla_{\theta_i} \ell_i, \quad \forall i \in \mathcal{T}_t, \quad (3)$$

where γ and γ_i are the step sizes for optimization. Since the task-specific classifiers are updated by their own gradients, we focus on the update of the shared backbone θ , and let the negative gradient $\mathbf{g}_i = -\nabla_{\theta} \ell_i$. However, treating each task equally in the AvgGrad method is naive and does not lead to Pareto optimality [12].

Definition 2 (Pareto Optimality).

- (1) (Pareto Dominate) Let θ_a, θ_b be two solutions for Problem (1), θ_a is said to dominate θ_b ($\theta_a \prec \theta_b$) if and only if $\ell_i(\theta_a) \leq \ell_i(\theta_b), \forall i \in \mathcal{T}$ and $\ell_i(\theta_a) < \ell_i(\theta_b), \exists i \in \mathcal{T}$.
- (2) (Pareto Critical) θ is called Pareto critical if no other solution in its neighborhood can have better values in all objective functions.
- (3) (Pareto Descent Direction) If θ_1 is not Pareto critical and can be updated to θ_2 by gradient $\mathbf{g}$. If $\theta_2 \prec \theta_1$, say $\mathbf{g}$ is a Pareto descent direction.

An elegant approach to achieving Pareto optimality in MOO is the Steepest Descent Method (SDM) [25], which identifies an optimal descent direction $\mathbf{d}^*$ by solving the following constrained optimization problem:

$$\begin{aligned} \mathbf{d}^*, \alpha^* = \arg \min_{\mathbf{d}, \alpha} \quad & \alpha + \frac{1}{2} \|\mathbf{d}\|^2, \\ \text{s.t.} \quad & \mathbf{g}_i^\top \mathbf{d} \leq \alpha, \quad \forall i \in \mathcal{T}, \end{aligned} \quad (4)$$

where the constraints ensure that the descent direction $\mathbf{d}$ does not conflict with the gradient of any task. Here, the Karush–Kuhn–Tucker (KKT) conditions are applied only to the local gradient aggregation subproblem in Eq. (4), whose goal is to compute a Pareto-oriented descent direction from the task gradients at the current iteration. They are not used to characterize the full deep neural network training objective as a globally convex problem. Based on this local subproblem, Eq. (4) can be transformed into the following dual form, which is solved by MGDA [61]:

$$\begin{aligned} \lambda^* = \arg \min_{\lambda} \quad & \left\| \sum_i \lambda_i \mathbf{g}_i \right\|^2, \\ \text{s.t.} \quad & \sum_i \lambda_i = 1 \text{ and } \lambda_i \geq 0, \forall i. \end{aligned} \quad (5)$$

The objective value of MGDA is 0 if and only if a KKT point is found. Otherwise, it yields a descent direction that simultaneously improves all task losses [23].

However, both AvgGrad and MGDA overlook the disparity in training dynamics between parallel tasks, which may differ significantly. As illustrated in Fig. 2, we conduct a toy PCL experiment using two synthetic functions¹. As observed,

¹The two functions are $f_1(x, y) = \log(1 + x^2) + 0.8(1 - e^x \cdot \sin(y))^2$ and $f_2(x, y) = \log(1 + y^2) + 0.004(0.1 + e^y \cdot \cos(x))^2$. The values of (x, y) are initialized to $(3, 3)$. The optimizer is the steepest gradient descent, and the step size is set to $2e-5$. The total number of iterations is 1,500. In PCL, we let $f_1(x, y)$ be the old task, and the new task $f_2(x, y)$ is set to access at the 500-th iteration.

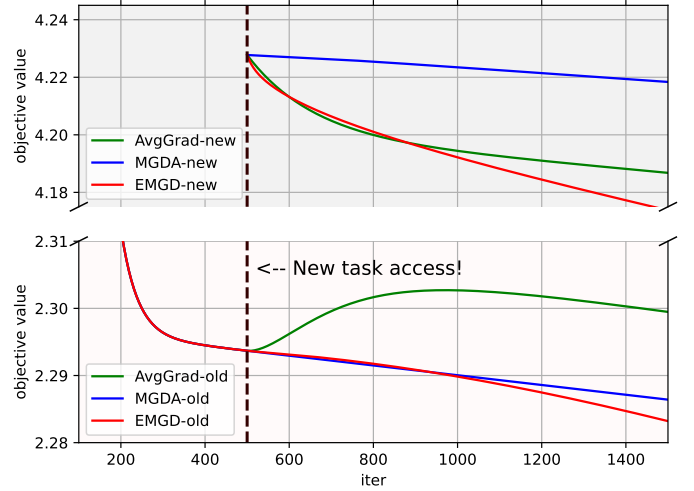


Fig. 2: Two synthetic functions are optimized in the PCL setting, where the new task arrives at iteration 500.

AvgGrad is biased toward the new task due to its larger gradient magnitudes, leading to suboptimal performance on the old task. Conversely, MGDA tends to produce a conservative Pareto descent direction, which may hinder the learning progress of the new task. To address this trade-off, we propose a novel yet simple method called *Elastic Multi-Gradient Descent* (EMGD) in the next section. EMGD accounts for the discrepancies in task-specific learning dynamics and computes a Pareto descent direction at each iteration that adaptively balances the optimization of newly accessed tasks and the retention of previously learned ones.

IV. ELASTIC MULTI-GRADIENT DESCENT

A. Elastic Constraint for Steepest Gradient Method

Instead of relying on direct weighting schemes or solving exact optimization problems, we seek a more controllable form of constraints tailored to the PCL setting. To this end, we introduce an elastic relaxation of Eq. (4) by defining a global elastic factor $\sigma > 0$ for the shared parameters θ . If θ is not Pareto critical, the following relaxed inequality holds:

$$\alpha(\theta) + \frac{1}{2} \|\mathbf{d}(\theta)\|^2 \leq \sigma \alpha^*(\theta), \quad (6)$$

where $\alpha^*(\theta)$ denotes the optimal value of Problem (4). When $\sigma = 1$, the inequality is satisfied only by the exact solution. To further incorporate directional alignment between each task gradient $\mathbf{g}_i$ and the descent direction $\mathbf{d}$, we generalize the elasticity on a per-task basis and reformulate Eq. (4) as follows:

$$\begin{aligned} \mathbf{d}^*, \alpha^* = \arg \min_{\mathbf{d}, \alpha} \quad & \alpha + \frac{1}{2} \|\mathbf{d}\|^2, \\ \text{s.t.} \quad & \mathbf{g}_i^\top \mathbf{d} \leq \sigma_i \alpha, \quad \forall i \in \mathcal{T}, \end{aligned} \quad (7)$$

where $\sigma_i \in (0, 1]$ is the task-specific elastic factor for task i . The only difference between Eq. (7) and the original SDM formulation lies in the introduction of these per-task relaxation parameters σ .

Lemma 1. Let $\mathbf{d}^*(\theta)$ and $\alpha^*(\theta)$ be the solution of Eq. (7) under the parameter θ . $\mathbf{d}^*(\theta)$ and $\alpha^*(\theta)$ hold the features: (1) if θ is Pareto critical, it has $\mathbf{d}^*(\theta) = \mathbf{0}$ and $\alpha^*(\theta) = 0$;

(2) if θ is not Pareto critical, it has $\alpha^*(\theta) \leq -\frac{1}{2}\|\mathbf{d}^*(\theta)\|^2 < 0$ and $\mathbf{g}_i^\top \mathbf{d}^*(\theta) \leq \sigma_i \alpha^*(\theta)$, $\forall i$.

Proof. We prove the two properties of Lemma 1 based on the definition of Pareto criticality in Definition 2.

Property 1): If θ is Pareto critical, then there exists no direction $\mathbf{d}$ such that all objective losses can be simultaneously improved. Hence, the optimal solution to Problem (7) is $\mathbf{d}^*(\theta) = \mathbf{0}$. Substituting into the constraints yields $\mathbf{g}_i^\top \mathbf{d}^*(\theta) = 0 \leq \sigma_i \alpha^*(\theta)$, implying $\alpha^*(\theta) = 0$.

Property 2): If the solution θ of PCL is not Pareto critical, it means that $\mathbf{d}(\theta) = \mathbf{0}$ and $\alpha(\theta) = 0$ are the feasible solutions to Problem (7). That is, we have

$$\alpha^*(\theta) + \frac{1}{2}\|\mathbf{d}^*(\theta)\|^2 < \alpha(\theta) + \frac{1}{2}\|\mathbf{d}(\theta)\|^2 = 0.$$

Thus,

$$\alpha^*(\theta) \leq \frac{1}{2}\|\mathbf{d}^*(\theta)\|^2 < 0.$$

Moreover, referring to the constraints, we have

$$\mathbf{g}_i^\top \mathbf{d}^*(\theta) \leq \sigma_i \alpha^*(\theta), \quad \forall i.$$

All claims of Lemma 1 are clear. ■

The task-specific elastic factors $\sigma_i \in (0, 1)$ introduce varying degrees of relaxation to the constraints for each task. Using the method of Lagrange multipliers for the linear inequality constraints in Eq. (7), the corresponding dual problem can be formulated as:

$$\begin{aligned} \lambda^* = \min_{\lambda} \quad & \left\| \sum_i \lambda_i \mathbf{g}_i \right\|^2, \\ \text{s.t.} \quad & \sum_i \lambda_i \sigma_i = 1 \quad \text{and} \quad \lambda_i \geq 0, \forall i. \end{aligned} \quad (8)$$

Compared to the primal formulation, the dual optimization takes place in a lower-dimensional decision space, with dimensionality equal to the number of currently active tasks, which is typically small in practice. As a standard quadratic programming (QP) problem, it can be solved efficiently using existing solvers. The primary challenge, however, lies in determining suitable elastic factors $\{\sigma_i\}$ for each task. Ideally, these factors should reflect the current learning dynamics: new tasks with larger gradients can benefit from looser constraints (i.e., higher σ_i), allowing faster adaptation, while old or memory-replayed tasks with smaller gradients are preserved through tighter constraints (i.e., lower σ_i), ensuring stability and preventing forgetting.

B. Evaluating Elastic Factor

To accommodate the diverse training dynamics in PCL, we aim to determine appropriate elastic factors for Problem (7) by considering two primary objectives: 1) *ensuring rapid convergence of newly introduced tasks*; 2) *minimizing the interference with previously learned tasks*. To assess how well a task satisfies both objectives, we introduce two quantitative metrics: *Gradient Magnitude Change* and *Gradient Similarity*.

(1) *Gradient Magnitude Change* (GMC): The magnitude of a task's gradient provides a proxy for its convergence status and vulnerability to interference from other tasks. We propose

Algorithm 1: PCL using EMGD

Input: Random-initialized θ , $\theta_{1:T}$; Step sizes γ , $\gamma_{1:T}$

Output: θ , $\theta_{1:T}$

```

1 for  $t$  in timeline do
2    $\mathcal{T}_t \leftarrow$  in-training task index;
3   for  $i \in \mathcal{T}_t$  do
4      $\mathcal{B}_i \sim \mathcal{D}_i$ ;
5      $\theta_i \leftarrow \theta_i - \gamma_i \nabla_{\theta_i} \ell_i(\mathcal{B}_i)$ ;
6      $\mathbf{g}_i = -\nabla_{\theta} \ell(\mathcal{B}_i)$ ;
7   end
8    $\mathcal{B}_m \sim \mathcal{M}$ ;
9    $\{\sigma_i | \forall i \in \mathcal{T}_t\} \leftarrow$  Compute elastic factors using
    GMC (Eq. (9)) or GS (Eq. (10));
10   $\lambda^* \leftarrow$  Solving Problem (8) with all  $\sigma$ ;
11   $\theta \leftarrow \theta + \gamma \cdot \sum_i \lambda_i \mathbf{g}_i$ ;
12  for  $\mathbf{x} \in \mathcal{B}_m$  do
13     $\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla_{\mathbf{x}} \|\mathbf{g}(\mathbf{x}) - \sum_i \lambda_i \mathbf{g}_i\|^2$ 
14  end
15 end

```

computing task-specific elastic factors based on the momentum-smoothed gradient norm:

$$\sigma_i = \frac{\exp(m_i)}{\sum_j \exp(m_j)}, \quad (9)$$

where m_i denotes the exponentially smoothed gradient norm for task i , defined as $m_i^t = \epsilon \cdot m_i^{t-1} + (1 - \epsilon) \cdot \|\mathbf{g}_i^t\|$, with a momentum coefficient ϵ empirically set to 0.9. This momentum-based smoothing helps stabilize the gradient norm estimation under mini-batch stochastic training.

(2) *Gradient Similarity* (GS): Gradient similarity quantifies the directional alignment between gradients of different tasks, serving as an indicator of inter-task interference. We define the elastic factor for task i based on the cosine similarities with all other task gradients:

$$\sigma_i = \frac{\exp(\sum_k \cos(g_i^t, g_k^t))}{\sum_j \exp(\sum_k \cos(g_j^t, g_k^t))}. \quad (10)$$

In contrast to GMC, which reflects only the temporal variation of a task's own gradient, GS incorporates the relational influence across tasks, offering a more global perspective on task interaction.

These two metrics offer complementary perspectives on task dynamics in the PCL setting. GMC focuses on the evolution of a task's own gradient magnitude, which reflects its convergence status and degree of interference from other tasks. A significant change in gradient norm often indicates that the task is either under-optimized or being disrupted, thus warranting greater learning attention. On the other hand, GS captures the alignment between task gradients, measuring whether a task's update is likely to interfere with or benefit others. By promoting gradients that align with the majority and suppressing conflicting ones, GS enables the model to coordinate learning across tasks more effectively. Together, these elastic factor strategies dynamically adjust task contributions during joint optimization, helping EMGD balance fast adaptation for new tasks with performance

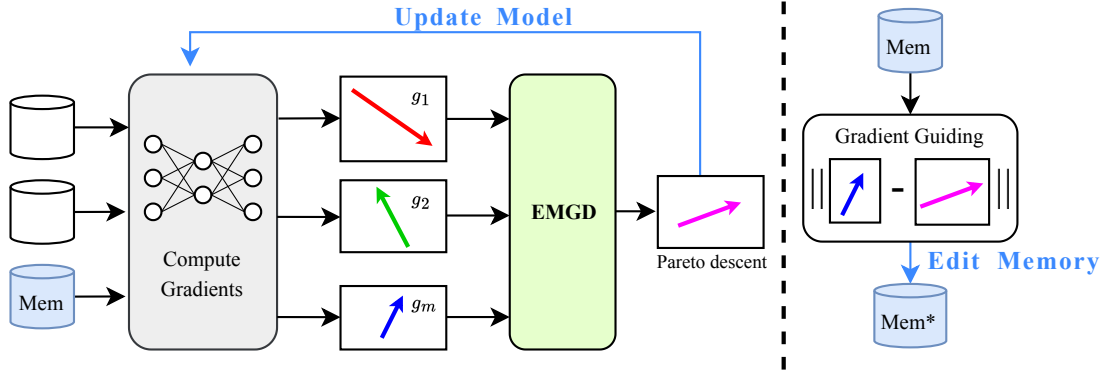


Fig. 3: An overview of the proposed method (one-step illustration). At each step, the activated data streams, along with the memory stream, are fed into the model to compute task-specific gradients. These gradients are then processed by EMGD to obtain a Pareto descent direction. The resulting optimal gradient is subsequently used to guide memory editing, ensuring reduced interference and improved task balance.

retention for old ones. We show the details of the full algorithm in Algorithm 1.

V. GRADIENT-GUIDED MEMORY EDITING

As discussed above, our proposed method, EMGD, aims to balance the training of old and new tasks by explicitly accounting for differences in gradient magnitudes and directional similarities. A critical challenge in rehearsal-based continual learning is *catastrophic forgetting*, which largely stems from the quality and utility of the stored memory samples. In standard SCL settings, rehearsal methods often assume that stored data points serve as accurate representatives of their respective tasks. However, this assumption may become increasingly invalid over time, leading to a gradual decline in the effectiveness of the memory buffer. To address this limitation, the Gradient-based Memory Editing (GMED) method [59] proposes refreshing memory samples by quantifying the interference between old and new tasks. Specifically, GMED evaluates the loss difference on the same memory data point $\mathbf{x}$ before and after model updates induced by new tasks.

Given a model with parameters θ , GMED constructs a reference model θ' updated with a task gradient $\mathbf{d}$. To reduce potential interference, the input sample $\mathbf{x}$ can be edited via:

$$\mathbf{x} = \mathbf{x} - \eta \cdot \nabla_{\mathbf{x}} \|\ell(\mathbf{x}, \theta) - \ell(\mathbf{x}, \theta')\|^2, \quad (11)$$

where $\theta' = \theta - \gamma \cdot \mathbf{d}$. While this strategy aims to minimize task interference, it has limitations: even if the loss difference is small, the corresponding gradients may point in significantly different directions. This misalignment renders loss-based editing an unreliable proxy for preserving gradient consistency.

In the PCL setting, where multiple data streams are active in parallel, such loss-based memory editing becomes even less reliable. To more accurately mitigate forgetting and task interference, we propose a novel memory editing strategy based on *gradient difference*. Rather than aligning loss values, we aim to directly align the memory sample's gradient with the optimal descent direction $\mathbf{d}$:

$$\mathbf{x} = \mathbf{x} - \eta \cdot \nabla_{\mathbf{x}} \|\mathbf{g}(\mathbf{x}) - \mathbf{d}\|^2, \quad (12)$$

where

$$\nabla_{\mathbf{x}} \|\mathbf{g}(\mathbf{x}) - \mathbf{d}\|^2 = 2 \left(\frac{\partial \ell(\mathbf{x}, \theta)}{\partial \theta} - \mathbf{d} \right)^\top \frac{\partial^2 \ell(\mathbf{x}, \theta)}{\partial \theta \partial \mathbf{x}}. \quad (13)$$

As shown in Algorithm 1 (L12-14) and Fig. 3, the proposed memory editing based on gradient guidance is easy to implement with a second-order derivative.

The objective in Eq. (12) encourages each memory sample $\mathbf{x}$ to be updated such that its gradient $\mathbf{g}(\mathbf{x}) = \nabla_{\theta} \ell(\mathbf{x}, \theta)$ aligns closely with the global descent direction $\mathbf{d}$ computed by EMGD. Intuitively, this process increases the compatibility between rehearsal samples and current optimization needs. Specifically, when a sample's gradient deviates from $\mathbf{d}$, the editing step pushes it toward a representation that better supports multi-task updates, thus reducing task interference. Unlike previous approaches (e.g., GMED [59]), which directly match task-specific losses under two parameter states, our formulation avoids dependency on a second forward-backward pass through θ' and instead leverages a geometrically grounded alignment term. The distance $\|\mathbf{g}(\mathbf{x}) - \mathbf{d}\|^2$ penalizes conflicting gradient directions and implicitly favors samples whose training signals are synergistic with current model optimization. As a result, the edited memories contribute more constructively during rehearsal, improving both retention and transfer across tasks.

VI. DISCUSSION

A. Convergence Analysis

In this subsection, we analyze the convergence behavior of the EMGD update rule through its accumulation points, and show that these accumulation points are Pareto critical under the proposed formulation.

Theorem 1. *The accumulation point of the continuous optimization by Algorithm 1 is Pareto critical.*

Proof. Let the solution at the k -th iteration be θ_k , and the corresponding objective of Problem (7) is $\alpha(\theta_k) + \frac{1}{2} \|\mathbf{d}(\theta_k)\|^2$. Suppose an accumulation point is $\hat{\theta}$ and

$$\lim_{k \rightarrow \infty} \ell(\theta_k) = \ell(\hat{\theta}). \quad (14)$$

Thus,

$$\lim_{k \rightarrow \infty} |\ell(\theta_{k+1}) - \ell(\theta_k)| = 0. \quad (15)$$

For each task, according to the Taylor Expansion, we have

$$\lim_{k \rightarrow \infty} \ell_i(\theta_{k+1}) - \ell_i(\theta_k) \leq \gamma \cdot \mathbf{g}_i^\top \mathbf{d}(\theta_k) \leq 0, \quad \forall i, \quad (16)$$

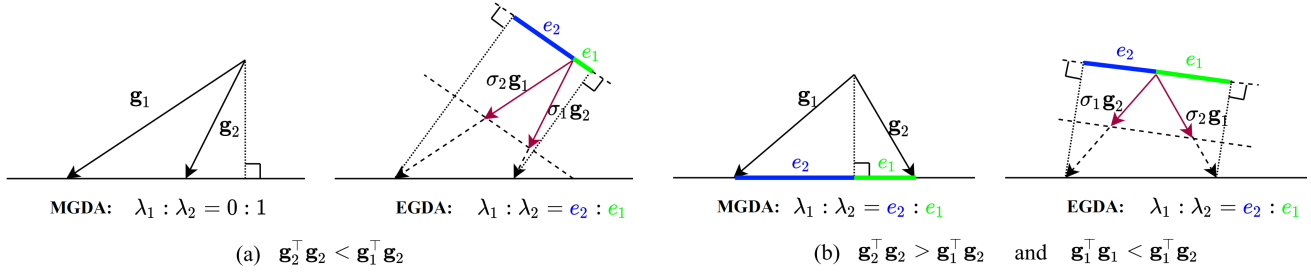


Fig. 4: Compared to MGDA [61], EMGD can control the ratio of λ_2 and λ_1 in a two-task scenario with imbalanced gradients.

where γ is the step size for optimization. Since $\ell_i(\theta_{k+1}) - \ell_i(\theta_k)$ serves as the lower bound of $\gamma \cdot \mathbf{g}_i^\top \mathbf{d}$, we have

$$\lim_{k \rightarrow \infty} \mathbf{g}_i^\top \mathbf{d}(\theta_k) = 0, \quad \forall i. \quad (17)$$

According to Lemma 1, we have $\mathbf{g}_i^\top \mathbf{d}(\theta_k) \leq \sigma_i \alpha(\theta_k)$ and $\alpha(\theta_k) \leq -\frac{1}{2} \|\mathbf{d}(\theta_k)\|^2$, thus

$$\lim_{k \rightarrow \infty} \alpha(\theta_k) = 0, \quad \lim_{k \rightarrow \infty} \mathbf{d}(\theta_k) = \mathbf{0}. \quad (18)$$

Since $\hat{\theta}$ is the accumulation point of θ^k , $\hat{\theta}$ is Pareto critical. ■

B. The Limitation of MGDA

Suppose we have two gradients $\mathbf{g}_1$ and $\mathbf{g}_2$ w.r.t. two tasks 1 and 2, and $\|\mathbf{g}_1\| > \|\mathbf{g}_2\|$. We build the objective of MGDA

$$\begin{aligned} \lambda_1^*, \lambda_2^* &= \min_{\lambda_1, \lambda_2} \|\lambda_1 \mathbf{g}_1 + \lambda_2 \mathbf{g}_2\|^2, \\ \text{s.t. } &\lambda_1 + \lambda_2 = 1 \text{ and } \lambda_i \geq 0, \forall i \in \{1, 2\}. \end{aligned} \quad (19)$$

It is easy to obtain the optimal λ_1^*, λ_2^* by solving the above problem, and we have

$$\lambda_1^* = \max \left(\min \left(\frac{\mathbf{g}_2^\top \mathbf{g}_2 - \mathbf{g}_1^\top \mathbf{g}_2}{\|\mathbf{g}_1 - \mathbf{g}_2\|^2}, 1 \right), 0 \right), \quad \lambda_2^* = 1 - \lambda_1^*. \quad (20)$$

It is easy to know that Eq. (20) can be piecewise as

$$\lambda_1^*, \lambda_2^* = \begin{cases} 1, 0, & \text{if } \mathbf{g}_1^\top \mathbf{g}_2 \geq \mathbf{g}_1^\top \mathbf{g}_1 \\ 0, 1, & \text{if } \mathbf{g}_1^\top \mathbf{g}_2 \geq \mathbf{g}_2^\top \mathbf{g}_2 \\ \frac{\mathbf{g}_2^\top \mathbf{g}_2 - \mathbf{g}_1^\top \mathbf{g}_2}{\|\mathbf{g}_1 - \mathbf{g}_2\|^2}, \frac{\mathbf{g}_1^\top \mathbf{g}_1 - \mathbf{g}_1^\top \mathbf{g}_2}{\|\mathbf{g}_1 - \mathbf{g}_2\|^2}, & \text{otherwise.} \end{cases} \quad (21)$$

Since $\|\mathbf{g}_1\| > \|\mathbf{g}_2\|$, we have $\mathbf{g}_1^\top \mathbf{g}_1 > \mathbf{g}_2^\top \mathbf{g}_2$ and $\mathbf{g}_1^\top \mathbf{g}_1 > \mathbf{g}_1^\top \mathbf{g}_2$.

If $\mathbf{g}_1^\top \mathbf{g}_1 > \mathbf{g}_1^\top \mathbf{g}_2 \geq \mathbf{g}_2^\top \mathbf{g}_2$, we have $\lambda_1^* = 0, \lambda_2^* = 1$ that $\lambda_1^* < \lambda_2^*$.

If $\mathbf{g}_1^\top \mathbf{g}_1 > \mathbf{g}_2^\top \mathbf{g}_2 \geq \mathbf{g}_1^\top \mathbf{g}_2$, we have $\lambda_1^* = \frac{\mathbf{g}_2^\top \mathbf{g}_2 - \mathbf{g}_1^\top \mathbf{g}_2}{\|\mathbf{g}_1 - \mathbf{g}_2\|^2}, \lambda_2^* = \frac{\mathbf{g}_1^\top \mathbf{g}_1 - \mathbf{g}_1^\top \mathbf{g}_2}{\|\mathbf{g}_1 - \mathbf{g}_2\|^2}$ that $\lambda_1^* < \lambda_2^*$.

To sum up, if we have $\|\mathbf{g}_1\| > \|\mathbf{g}_2\|$ in MGDA, then $\lambda_1 < \lambda_2$ and the update $\mathbf{d} = -\lambda_1 \mathbf{g}_1 - \lambda_2 \mathbf{g}_2$ has preference towards the small gradient. And this preference depends on the gradient norm difference. A similar phenomenon can be generalized to multiple tasks. ■

C. Comparing MGDA with EMGD

We now illustrate how EMGD effectively controls the Pareto descent direction, in contrast to MGDA. Consider the scenario involving two tasks at a particular training step, where the gradients $\mathbf{g}_1$ and $\mathbf{g}_2$ are obtained, and $\|\mathbf{g}_1\| > \|\mathbf{g}_2\|$. Under MGDA, solving Eq. (5) yields $\lambda_1 < \lambda_2$, implying that MGDA inherently favors preserving the objective of the task with the smaller gradient norm. This bias arises because MGDA balances gradients based on magnitudes, without considering the training dynamics or relative task priorities. In contrast, EMGD introduces task-specific elastic factors to modulate the optimization. The two-task EMGD objective becomes:

$$\begin{aligned} \lambda_1^*, \lambda_2^* &= \arg \min_{\lambda_1, \lambda_2} \|\lambda_1 \mathbf{g}_1 + \lambda_2 \mathbf{g}_2\|^2, \\ \text{s.t. } &\lambda_1 \sigma_1 + \lambda_2 \sigma_2 = 1, \lambda_1 \geq 0, \lambda_2 \geq 0. \end{aligned} \quad (22)$$

If $\sigma_1 \mathbf{g}_2^\top \mathbf{g}_2 < \sigma_2 \mathbf{g}_1^\top \mathbf{g}_2$, we have $\lambda_1^* = 0$ and $\lambda_2^* = \frac{1}{\sigma_2}$. If $\sigma_2 \mathbf{g}_1^\top \mathbf{g}_1 < \sigma_1 \mathbf{g}_1^\top \mathbf{g}_2$, we have $\lambda_1^* = \frac{1}{\sigma_1}$ and $\lambda_2^* = 0$. Otherwise,

$$\lambda_1^* = \frac{\sigma_1 \mathbf{g}_2^\top \mathbf{g}_2 - \sigma_2 \mathbf{g}_1^\top \mathbf{g}_2}{\|\sigma_2 \mathbf{g}_1 - \sigma_1 \mathbf{g}_2\|^2}, \quad \lambda_2^* = \frac{\sigma_2 \mathbf{g}_1^\top \mathbf{g}_1 - \sigma_1 \mathbf{g}_1^\top \mathbf{g}_2}{\|\sigma_2 \mathbf{g}_1 - \sigma_1 \mathbf{g}_2\|^2}. \quad (23)$$

In this situation,

$$\frac{\lambda_1^*}{\lambda_2^*} = \frac{\sigma_1 \mathbf{g}_2^\top \mathbf{g}_2 - \sigma_2 \mathbf{g}_1^\top \mathbf{g}_2}{\sigma_2 \mathbf{g}_1^\top \mathbf{g}_1 - \sigma_1 \mathbf{g}_1^\top \mathbf{g}_2} = \frac{\|\mathbf{g}_2\| \cos \langle \sigma_2 \mathbf{g}_1 - \sigma_1 \mathbf{g}_2, \mathbf{g}_2 \rangle}{\|\mathbf{g}_1\| \cos \langle \sigma_1 \mathbf{g}_2 - \sigma_2 \mathbf{g}_1, \mathbf{g}_1 \rangle}. \quad (24)$$

As illustrated in Fig. 4, the ratio between λ_1^* and λ_2^* corresponds to the ratio of the projections of the task gradients $\mathbf{g}_1$ and $\mathbf{g}_2$ onto the direction $\sigma_2 \mathbf{g}_1 - \sigma_1 \mathbf{g}_2$. MGDA has $\sigma_1 = \sigma_2 = 1$, and the ratio is governed by the relative magnitudes of the gradients, potentially leading to an imbalance that favors tasks with smaller gradients. In contrast, EMGD introduces task-specific elastic factors to adaptively modulate this ratio, enabling a more balanced and controllable optimization across tasks.

D. Spatiotemporal Complexity Analysis

Let N be the number of active tasks, P the number of model parameters, M the number of rehearsal samples, and d the input dimension. The computational complexity of Algorithm 1 is composed of four main parts: (1) Gradient Computation: Calculating gradients for N tasks over the shared model requires $\mathcal{O}(N \cdot P)$. (2) Elastic Factor Computation: Using Gradient Magnitude Change (GMC) costs $\mathcal{O}(N \cdot P)$, while Gradient Similarity (GS) involves pairwise cosine similarity, resulting in $\mathcal{O}(N^2 \cdot P)$. (3) Quadratic Programming (QP): Solving the dual-form QP with N variables incurs $\mathcal{O}(N^3)$ complexity. (4) Memory Editing: Editing M rehearsal samples

based on gradient guidance yields $\mathcal{O}(M \cdot P \cdot d)$. Overall, the total time complexity is $\mathcal{O}(N \cdot P + N^3 + M \cdot P \cdot d)$ for GMC, and $\mathcal{O}(N^2 \cdot P + N^3 + M \cdot P \cdot d)$ for GS. Space complexity includes storing N task gradients ($\mathcal{O}(N \cdot P)$) and M rehearsal samples ($\mathcal{O}(M \cdot d)$). In summary, although EMGD introduces additional overhead from multi-objective optimization and memory editing, this is offset by its benefits in learning stability and mitigating task interference.

VII. EXPERIMENT

A. Experimental Setup

Datasets. We adopt three datasets for the PCL setting following MaxDO [50] for evaluation: (1) *PS-EMNIST*: The 62-class EMNIST dataset is divided into 5 tasks, with the first three tasks containing 12 classes each and the last two containing 13 classes each. (2) *PS-CIFAR-100*: The 100 classes are split into 20 tasks with 5 classes per task. (3) *PS-ImageNet-TINY*: A 200-class subset of ImageNet is split into 20 tasks, each with 10 randomly selected classes. For each task, we randomly generate 3 label sets and 3 corresponding timelines, resulting in 9 variants per task. To ensure fair comparison, we use 3 fixed random seeds (1,234 to 1,236), leading to 27 independent experimental settings per dataset. In all experiments, each data subset is used only once and is discarded afterward. We evaluate performance under two types of continual learning scenarios: 1) *Task-Incremental Learning*, where task identities are available during both training and testing; 2) *Class-Incremental Learning*, where task identities are withheld during testing, presenting a more challenging setting.

Implementation details. The input batch size is set to 128 for all datasets. For the backbone architecture, we adopt a two-layer MLP for PS-EMNIST, and ResNet-18 [31] for both PS-CIFAR-100 and PS-ImageNet-TINY. In the rehearsal strategy, we store a fixed number of exemplars per class: 5 for PS-EMNIST, 30 for PS-CIFAR-100, and 15 for PS-ImageNet-TINY.

Evaluation metrics. To evaluate PCL performance, we follow common metrics used in prior continual learning studies [49], [16], [59], namely the final average accuracy and forgetting. Let $\bar{e} = \max(e_1, e_2, \dots, e_T)$ denote the final training time. The two metrics are computed as:

$$A_{\bar{e}} = \frac{1}{T} \sum_{t=1}^T a_{\bar{e}}^t, \quad F_{\bar{e}} = \frac{1}{T} \sum_{t=1}^T (a_{\bar{e}}^t - a_{e_t}^t). \quad (25)$$

Here, a_k^t denotes the testing accuracy on task t at time k . $A_{\bar{e}}$ measures the final average accuracy across all tasks, while $F_{\bar{e}}$ (also known as backward transfer) quantifies the average forgetting, computed as the performance drop between the time each task was learned and the end of training.

B. Major Comparisons with the SOTAs

We compare EMGD with representative methods from three categories: MTL, SCL, and PCL. For MTL, we evaluate MGDA [61], GradNorm [18], DWA [45], GradDrop [19], PCGrad [75], RLW [41], Nash-MTL [56], Aligned-MTL [62], CREST [34], and AvgGrad (see Sec. III-B). These methods, originally designed for simultaneous task training, are adapted to PCL by treating each time step as an MTL subunit

and training sequentially across the subunits. For SCL, we consider AGEM [16], GMED [59], MDMTR [51], CBRs [20], CoPE [22], AOP [28], LODE [38], OnPro [69], MIND [9], NsCE [71], and RanDumb [57]. We adapt them to PCL by mixing one batch from each active task with one from the memory buffer, simulating naive parallel training. For PCL, we include MaxDO [50] as a strong baseline. Our method is evaluated in two variants: EMGD(GMC) and EMGD(GS), which estimate task-specific elastic factors using gradient magnitude change and gradient similarity, respectively, to guide memory editing and gradient alignment.

As shown in Tables I and II, we compare EMGD with MTL and SCL baselines on three datasets under both task-incremental and class-incremental PCL settings. Key observations include: (1) *Consistent performance gains*: EMGD achieves the highest final accuracy ($A_{\bar{e}}$) across all benchmarks, validating its effectiveness in alleviating task interference and catastrophic forgetting. (2) *Impact of memory editing (ME)*: Incorporating gradient-guided ME further improves performance, indicating that leveraging gradient signals to refine memory content enhances stability and retention during training. (3) *Task vs. class-incremental PCL*: Task-incremental settings show less forgetting due to the availability of task IDs, which mitigate knowledge interference. In contrast, class-incremental settings lack such cues, making forgetting more severe despite similar levels of training conflict. (4) *Good baseline of AvgGrad*: The surprisingly competitive results of AvgGrad reveal that simple gradient averaging can be effective in PCL. This exposes a limitation of many MTL/SCL methods, which often optimize task balance or memory alone, failing to address both jointly as required in PCL. (5) *MGDA's varied performance*: MGDA performs worse in task-incremental PCL due to conflict sensitivity, but is competitive in class-incremental settings where preserving past knowledge is more crucial. This contrast highlights the differing priorities between the two settings. (6) *GMC vs. GS*: In task-incremental PCL, GS generally outperforms GMC due to finer control over gradient structures enabled by task IDs. Conversely, GMC yields better results in class-incremental PCL, where modeling inter-task relationships compensates for the lack of task identity. These findings suggest that elastic factor design should be scenario-aware. Overall, EMGD effectively addresses the dual challenges of conflict and forgetting in PCL by jointly optimizing gradient alignment and memory usage.

C. Analysis on Elastic Factors

Effect of changing elastic factors. To gain deeper insight into how the proposed elastic factor mechanisms affect training dynamics, we design a controlled two-task experiment based on PS-CIFAR-10. As illustrated in Fig. 5(a), the training process begins with Task 1, and Task 2 is introduced after one-third of the total iterations. We conduct multiple experiments as follows. (1) *Test Accuracy* (Fig. 5(b)): After Task 2's arrival, both EMGD(GMC) and EMGD(GS) significantly outperform AvgGrad in overall accuracy. They adapt better to the new task while preserving performance on Task 1, validating the importance of dynamic gradient modulation. (2) *Loss Curves*

TABLE I: Task-incremental PCL final performance comparisons (avg $\pm$ std). **Red bold** indicates the best result, **Blue bold** denotes the second-best result (numerically), and Underline highlights results statistically comparable to the best (based on overlapping standard deviations). We adopt a consistent result formatting convention across all performance tables.

Method (Rehearsal)	Venue	Type	PS-EMNIST		PS-CIFAR-100		PS-ImageNet-TINY	
			A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$
AvgGrad	/	MTL	89.344 $\pm$ 0.231	-5.287 $\pm$ 0.216	47.579 $\pm$ 0.089	23.691 $\pm$ 0.432	38.392 $\pm$ 0.076	0.764 $\pm$ 0.139
MGDA [23]	NeurIPS'2018	MTL	84.887 $\pm$ 0.469	<u>-4.301$\pm$0.818</u>	48.957 $\pm$ 0.451	25.088 $\pm$ 0.203	38.058 $\pm$ 0.740	1.465 $\pm$ 0.490
GradNorm [18]	ICML'2018	MTL	87.888 $\pm$ 0.158	-6.197 $\pm$ 0.183	47.210 $\pm$ 1.323	23.474 $\pm$ 0.688	38.226 $\pm$ 0.769	1.647 $\pm$ 0.667
DWA [45]	CVPR'2019	MTL	88.405 $\pm$ 0.322	-5.452 $\pm$ 0.338	44.969 $\pm$ 0.378	22.682 $\pm$ 0.537	34.290 $\pm$ 1.099	-1.053 $\pm$ 0.866
PCGrad [75]	NeurIPS'2020	MTL	89.698 $\pm$ 0.164	-4.921 $\pm$ 0.121	47.026 $\pm$ 0.538	23.244 $\pm$ 0.740	39.427 $\pm$ 1.275	2.017 $\pm$ 0.769
RLW [41]	Arxiv'2021	MTL	89.288 $\pm$ 0.218	-5.226 $\pm$ 0.204	47.574 $\pm$ 0.349	23.833 $\pm$ 0.117	38.531 $\pm$ 1.610	1.332 $\pm$ 1.148
Nash-MTL [56]	ICML'2022	MTL	89.132 $\pm$ 0.135	-5.778 $\pm$ 0.474	49.629 $\pm$ 0.414	20.915 $\pm$ 0.735	39.569 $\pm$ 0.175	4.776 $\pm$ 0.895
Aligned-MTL [62]	CVPR'2023	MTL	87.010 $\pm$ 0.574	4.977 $\pm$ 0.407	51.446 $\pm$ 0.011	25.023 $\pm$ 0.162	36.153 $\pm$ 0.949	1.463 $\pm$ 0.166
CREST [34]	AAAI'2024	MTL	88.419 $\pm$ 0.347	4.885 $\pm$ 0.303	50.143 $\pm$ 0.046	25.983 $\pm$ 0.431	38.592 $\pm$ 0.884	1.443 $\pm$ 0.426
AGEM [16]	ICLR'2019	SCL	87.022 $\pm$ 0.519	-7.646 $\pm$ 0.483	27.379 $\pm$ 0.585	5.416 $\pm$ 0.851	28.530 $\pm$ 0.994	-7.070 $\pm$ 1.410
GMED [59]	NeurIPS'2019	SCL	85.471 $\pm$ 0.324	-8.595 $\pm$ 0.335	49.094 $\pm$ 1.792	18.356 $\pm$ 1.345	34.495 $\pm$ 1.568	-0.640 $\pm$ 1.799
ER [17]	Arxiv'2019	SCL	89.106 $\pm$ 0.315	-5.525 $\pm$ 0.207	47.324 $\pm$ 0.584	23.330 $\pm$ 0.762	35.950 $\pm$ 0.763	-0.767 $\pm$ 0.591
DER++ [11]	NeurIPS'2020	SCL	89.654 $\pm$ 0.451	-4.979 $\pm$ 0.293	49.276 $\pm$ 0.480	22.634 $\pm$ 0.165	40.039 $\pm$ 0.039	2.601 $\pm$ 0.643
CBRS [20]	ICML'2020	SCL	82.141 $\pm$ 0.667	-10.783 $\pm$ 0.816	45.729 $\pm$ 0.275	23.336 $\pm$ 0.677	32.135 $\pm$ 0.603	1.950 $\pm$ 0.047
MDMTR [51]	AAAI'2021	SCL	73.115 $\pm$ 0.382	-16.359 $\pm$ 0.178	38.865 $\pm$ 1.691	18.297 $\pm$ 1.052	23.422 $\pm$ 0.273	-1.544 $\pm$ 0.389
CoPE [22]	ICCV'2021	SCL	85.179 $\pm$ 0.807	-9.412 $\pm$ 0.502	46.057 $\pm$ 0.908	23.978 $\pm$ 0.067	37.491 $\pm$ 0.312	2.550 $\pm$ 0.817
AOP [28]	AAAI'2022	SCL	88.820 $\pm$ 0.768	-6.523 $\pm$ 0.154	45.628 $\pm$ 0.561	24.611 $\pm$ 0.192	38.901 $\pm$ 0.198	3.002 $\pm$ 0.820
LODE [38]	NeurIPS'2023	SCL	74.547 $\pm$ 0.413	-15.627 $\pm$ 0.376	40.180 $\pm$ 0.754	20.917 $\pm$ 0.193	26.179 $\pm$ 0.134	2.241 $\pm$ 0.183
OnPro [69]	ICCV'2023	SCL	84.112 $\pm$ 0.157	-8.887 $\pm$ 0.378	47.294 $\pm$ 0.912	22.094 $\pm$ 0.972	34.910 $\pm$ 0.241	2.843 $\pm$ 0.441
MIND [9]	AAAI'2024	SCL	78.556 $\pm$ 0.417	-14.495 $\pm$ 0.781	48.900 $\pm$ 0.837	24.580 $\pm$ 0.903	28.332 $\pm$ 0.902	1.406 $\pm$ 0.329
NsCE [71]	NeurIPS'2024	SCL	81.747 $\pm$ 0.104	-8.237 $\pm$ 0.706	44.814 $\pm$ 0.577	23.184 $\pm$ 0.495	31.807 $\pm$ 0.475	1.294 $\pm$ 0.253
RanDumb [57]	arXiv'2024	SCL	88.527 $\pm$ 0.468	-7.594 $\pm$ 0.252	47.416 $\pm$ 0.312	22.682 $\pm$ 0.393	39.074 $\pm$ 0.428	2.868 $\pm$ 0.584
MaxDO [50]	ICCV'2023	PCL	90.189 $\pm$ 0.314	-4.258 $\pm$ 0.311	50.203 $\pm$ 0.978	24.510 $\pm$ 0.092	40.770 $\pm$ 0.354	3.119 $\pm$ 0.450
EMGD(GMC)	Ours	PCL	90.442 $\pm$ 0.144	-4.142 $\pm$ 0.182	52.798 $\pm$ 0.768	26.858 $\pm$ 0.426	46.666 $\pm$ 0.095	8.934 $\pm$ 0.350
EMGD(GMC)+ME	Ours	PCL	90.402 $\pm$ 0.102	-4.217 $\pm$ 0.093	56.611 $\pm$ 0.102	23.970 $\pm$ 0.424	48.746 $\pm$ 0.413	6.086 $\pm$ 0.499
EMGD(GS)	Ours	PCL	90.348 $\pm$ 0.157	-4.265 $\pm$ 0.187	52.389 $\pm$ 1.109	26.530 $\pm$ 0.524	46.213 $\pm$ 0.334	7.830 $\pm$ 0.715
EMGD(GS)+ME	Ours	PCL	90.524 $\pm$ 0.038	-4.501 $\pm$ 0.060	56.378 $\pm$ 0.244	22.760 $\pm$ 0.399	49.269 $\pm$ 0.285	7.744 $\pm$ 0.481

(Fig. 5(c)): The new task (Task 2) exhibits faster convergence under EMGD, while the loss for Task 1 remains stable. This indicates that elastic factor-based reweighting accelerates learning for new tasks without destabilizing existing ones. (3) *Elastic Factor Trajectories* (Fig. 5(d)): The evolution of task-specific weights σ_1 shows that when Task 2 arrives, the model initially assigns a higher weight to the new task, facilitating rapid adaptation. Over time, the weights stabilize, reflecting the model's ability to balance learning needs dynamically. (4) *Gradient Similarity* (Fig. 5(e)): The cosine similarity between task gradients reveals that EMGD consistently maintains gradient angles below 90° , meaning that it effectively reduces gradient conflict and promotes cooperative updates. (5) *Gradient Norm Alignment* (Fig. 5(f)): Using the normalized magnitude difference metric $\text{mag}(\mathbf{g}_1, \mathbf{g}_2) = \frac{2\|\mathbf{g}_1\|_2\|\mathbf{g}_2\|_2}{\|\mathbf{g}_1\|_2^2 + \|\mathbf{g}_2\|_2^2}$, we observe that EMGD maintains better alignment in gradient norms, suggesting balanced optimization pressure across tasks. (6) *Update Magnitude* (Fig. 5(g)): We visualize the norm of the shared update direction, i.e., $\|\mathbf{d}(\theta_k)\|_2$. After Task 2 arrives, all methods exhibit a transient increase, as the newly introduced task perturbs the shared optimization process. However, EMGD recovers more quickly and returns to a noticeably smaller and more stable regime than the competing baselines, indicating stronger robustness to task-induced perturbations and a more stable Pareto-oriented update process. (7) *Directional Residual* (Fig. 5(h)): We further report the residual $r_k = \max_i |\mathbf{g}_i^\top \mathbf{d}(\theta_k)|$, which summarizes the largest directional discrepancy between the update direction and the task gradients. EMGD reduces this quantity more quickly after Task 2 arrives and maintains a lower level thereafter than the competing methods, suggesting better restoration of a Pareto-

oriented update state. Overall, these additional visualizations show that both GMC and GS contribute meaningfully to stabilizing multi-task learning dynamics in PCL. The adaptive weighting induced by elastic factors helps the model absorb task-induced disturbances and return more quickly to a smaller and more stable update regime. Fig. 6 extends the learning-stage analysis from the controlled two-task setting to PS-EMNIST with five sequentially arriving tasks. Similar to the two-task case, repeated task arrivals introduce transient spikes in both the update magnitude and the directional residual. Nevertheless, EMGD consistently shows faster recovery and smaller long-term fluctuations than the baselines, indicating that the favorable optimization behavior of EMGD remains observable under repeated task arrivals in a larger benchmark scenario.

Comparisons with other factor calculations. We conduct an ablation study on PS-CIFAR-100 under both task- and class-incremental settings to evaluate different strategies for computing elastic gradient factors. Compared methods include three naive baselines: (1) *Uniform*, where each task is assigned an equal factor ($\sigma_i = 1/N$), equivalent to simple gradient averaging. (2) *Random*, where each task receives a fixed random scalar from $(0, 1)$. (3) *Loss-Reverse*, where each task's factor is inversely proportional to its current loss ($\sigma_i = 1/L_i$), a naive but commonly assumed heuristic. We further compare with gradient-based weighting baselines: MGDA [61], GradNorm [18], PCGrad [75], and MaxDO [50]. To isolate the effect of elastic factor design, all results are reported *without memory editing* (w/o ME). As shown in Table III, both GMC- and GS-based factors outperform all baselines, with GMC slightly better under the task-incremental setting. These results validate the effectiveness of our gradient-based elastic metrics and highlight their key contribution to the success of EMGD.

TABLE II: Class-incremental PCL final performance comparisons (avg $\pm$ std).

Method (Rehearsal)	Venue	Type	PS-EMNIST		PS-CIFAR-100		PS-ImageNet-TINY	
			A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$
AvgGrad	NeurIPS'2018	MTL	43.823 $\pm$ 0.566	-44.484 $\pm$ 0.596	8.921 $\pm$ 0.276	-4.985 $\pm$ 0.702	7.511 $\pm$ 0.464	-15.825 $\pm$ 0.228
MGDA [23]		MTL	52.823 $\pm$ 0.201	-7.753 $\pm$ 0.661	11.323 $\pm$ 0.282	-4.065 $\pm$ 0.725	8.318 $\pm$ 0.155	-13.118 $\pm$ 0.739
GradNorm [18]	ICML'2018	MTL	43.292 $\pm$ 0.590	-44.824 $\pm$ 0.688	9.477 $\pm$ 0.222	-4.403 $\pm$ 1.236	6.395 $\pm$ 0.130	-15.869 $\pm$ 0.336
DWA [45]	CVPR'2019	MTL	42.734 $\pm$ 0.211	-38.944 $\pm$ 0.581	3.519 $\pm$ 0.116	-7.470 $\pm$ 0.244	3.003 $\pm$ 0.139	-12.774 $\pm$ 0.158
PCGrad [75]	NeurIPS'2020	MTL	45.035 $\pm$ 0.273	-43.309 $\pm$ 0.176	10.140 $\pm$ 0.253	-5.149 $\pm$ 0.553	7.789 $\pm$ 0.237	-15.860 $\pm$ 0.318
RLW [41]	Arxiv'2021	MTL	44.595 $\pm$ 0.480	-40.424 $\pm$ 0.325	9.957 $\pm$ 0.135	-5.173 $\pm$ 0.613	7.419 $\pm$ 0.258	-16.079 $\pm$ 0.255
Nash-MTL [56]	ICML'2022	MTL	45.141 $\pm$ 0.322	-40.931 $\pm$ 0.128	9.831 $\pm$ 0.483	-5.901 $\pm$ 0.283	6.290 $\pm$ 0.741	-15.832 $\pm$ 0.775
Aligned-MTL [62]	CVPR'2023	MTL	49.414 $\pm$ 0.539	-28.843 $\pm$ 0.791	12.108 $\pm$ 0.156	-0.051 $\pm$ 0.593	8.896 $\pm$ 0.155	-16.293 $\pm$ 0.336
CREST [34]	AAAI'2024	MTL	50.078 $\pm$ 0.219	-29.586 $\pm$ 0.227	11.726 $\pm$ 0.714	-0.993 $\pm$ 0.418	8.278 $\pm$ 0.124	-15.164 $\pm$ 0.139
AGEM [16]	ICLR'2019	SCL	26.249 $\pm$ 0.511	-62.480 $\pm$ 0.527	3.374 $\pm$ 0.098	-10.462 $\pm$ 0.543	3.379 $\pm$ 0.149	-18.514 $\pm$ 0.516
GMED [59]	NeurIPS'2019	SCL	22.694 $\pm$ 0.153	-65.805 $\pm$ 0.255	4.941 $\pm$ 0.407	-12.837 $\pm$ 0.710	3.386 $\pm$ 0.348	-19.060 $\pm$ 0.682
ER [17]	Arxiv'2019	SCL	43.474 $\pm$ 0.860	-44.597 $\pm$ 0.861	9.317 $\pm$ 0.339	-5.337 $\pm$ 0.626	6.126 $\pm$ 0.277	-13.686 $\pm$ 0.290
DER++ [11]	NeurIPS'2020	SCL	50.039 $\pm$ 0.520	-43.422 $\pm$ 0.439	11.022 $\pm$ 0.642	-1.653 $\pm$ 0.120	9.190 $\pm$ 0.429	-14.049 $\pm$ 0.227
CBRS [20]	ICML'2020	SCL	39.447 $\pm$ 0.418	-40.499 $\pm$ 0.730	6.251 $\pm$ 0.914	-4.372 $\pm$ 0.710	7.190 $\pm$ 0.561	-14.186 $\pm$ 0.891
MDMTR [51]	AAAI'2021	SCL	42.199 $\pm$ 1.100	-32.140 $\pm$ 0.996	5.525 $\pm$ 0.260	-3.454 $\pm$ 0.145	2.793 $\pm$ 0.042	-12.992 $\pm$ 0.082
CoPE [22]	ICCV'2021	SCL	41.462 $\pm$ 0.191	-41.810 $\pm$ 0.355	9.510 $\pm$ 0.284	-3.180 $\pm$ 0.199	6.296 $\pm$ 0.435	-15.593 $\pm$ 0.132
AOP [28]	AAAI'2022	SCL	50.282 $\pm$ 0.478	-42.790 $\pm$ 0.094	11.065 $\pm$ 0.328	0.504 $\pm$ 0.193	10.931 $\pm$ 0.812	-13.641 $\pm$ 0.183
LODE [38]	NeurIPS'2023	SCL	46.060 $\pm$ 0.312	-40.388 $\pm$ 0.735	11.681 $\pm$ 0.571	-2.466 $\pm$ 0.887	9.671 $\pm$ 0.012	-13.543 $\pm$ 0.843
OnPro [69]	ICCV'2023	SCL	45.099 $\pm$ 0.934	-39.253 $\pm$ 0.277	11.372 $\pm$ 0.510	-1.630 $\pm$ 0.828	9.187 $\pm$ 0.486	-13.237 $\pm$ 0.150
MIND [9]	AAAI'2024	SCL	47.675 $\pm$ 0.198	-35.299 $\pm$ 0.091	11.310 $\pm$ 0.056	-0.750 $\pm$ 0.030	9.273 $\pm$ 0.378	-12.349 $\pm$ 0.189
NsCE [71]	NeurIPS'2024	SCL	40.874 $\pm$ 0.473	-36.031 $\pm$ 0.485	6.810 $\pm$ 0.310	-4.070 $\pm$ 0.433	8.274 $\pm$ 0.274	-14.449 $\pm$ 0.972
RanDumb [57]	arXiv'2024	SCL	44.836 $\pm$ 0.791	-34.419 $\pm$ 0.104	10.289 $\pm$ 0.915	-2.092 $\pm$ 0.855	7.155 $\pm$ 0.873	-13.734 $\pm$ 0.416
MaxDO [50]	ICCV'2023	PCL	53.139 $\pm$ 0.156	-11.903 $\pm$ 0.476	12.237 $\pm$ 0.176	-2.280 $\pm$ 0.270	9.532 $\pm$ 0.363	-12.511 $\pm$ 0.610
EMGD(GMC)	Ours	PCL	53.441 $\pm$ 0.341	-13.250 $\pm$ 0.904	13.705 $\pm$ 0.633	0.367 $\pm$ 1.299	11.512 $\pm$ 0.216	-11.748 $\pm$ 0.331
EMGD(GMC)+ME	Ours	PCL	56.891 $\pm$ 0.062	-18.515 $\pm$ 0.211	16.362 $\pm$ 0.294	-1.654 $\pm$ 0.808	12.641 $\pm$ 0.921	-9.890 $\pm$ 0.743
EMGD(GS)	Ours	PCL	53.425 $\pm$ 0.279	-25.981 $\pm$ 0.587	13.601 $\pm$ 0.374	1.530 $\pm$ 0.524	11.588 $\pm$ 0.508	-11.324 $\pm$ 1.072
EMGD(GS)+ME	Ours	PCL	56.031 $\pm$ 0.254	-17.826 $\pm$ 0.296	16.189 $\pm$ 1.343	-2.937 $\pm$ 1.490	12.796 $\pm$ 0.398	-9.093 $\pm$ 0.831

TABLE III: Impact of factor calculation on PS-CIFAR-100.

Method	Task-incremental PCL		Class-incremental PCL	
	A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$
Uniform	47.579 $\pm$ 0.089	23.691 $\pm$ 0.432	8.921 $\pm$ 0.276	-4.985 $\pm$ 0.702
Random	44.986 $\pm$ 0.509	24.157 $\pm$ 0.448	9.650 $\pm$ 0.891	-6.230 $\pm$ 0.278
Loss-Reverse	48.302 $\pm$ 0.887	24.544 $\pm$ 0.281	10.704 $\pm$ 0.359	-4.858 $\pm$ 0.716
MGDA	48.957 $\pm$ 0.451	25.088 $\pm$ 0.203	11.323 $\pm$ 0.282	-4.065 $\pm$ 0.725
GradNorm	47.210 $\pm$ 1.323	23.474 $\pm$ 0.688	9.477 $\pm$ 0.222	-4.403 $\pm$ 1.236
PCGrad	47.026 $\pm$ 0.538	23.244 $\pm$ 0.740	10.140 $\pm$ 0.253	-5.149 $\pm$ 0.553
MaxDO	50.203 $\pm$ 0.978	24.510 $\pm$ 0.092	12.237 $\pm$ 0.176	-2.280 $\pm$ 0.270
GMC (w/o ME)	52.798 $\pm$ 0.768	26.858 $\pm$ 0.426	13.705 $\pm$ 0.633	0.367 $\pm$ 1.299
GS (w/o ME)	52.389 $\pm$ 1.109	26.530 $\pm$ 0.524	13.601 $\pm$ 0.374	1.530 $\pm$ 0.524

D. Analysis on Rehearsal and Memory Editing

Different memory size. We investigate the effect of varying rehearsal memory sizes on PCL performance using the PS-CIFAR-100 dataset. Results are shown in Fig. 7. We observe that increasing memory size consistently improves final accuracy (A_e), indicating that replaying stored data effectively mitigates catastrophic forgetting in PCL. However, the rate of performance improvement diminishes as the memory size grows, and in some cases, excessive memory may even lead to slight performance degradation, likely due to redundancy or overfitting to historical data. Moreover, we observe that EMGD with GS benefits more from larger memory sizes, showing greater improvement, whereas EMGD with GMC performs better in low-memory regimes. This highlights their respective strengths: GS excels with abundant data, while GMC is more efficient when memory is limited.

Visualization of edited gradient. In Fig. 8, we conduct visualization on gradient editing (normalized gradients in Eq. (12)) using the three datasets. Brighter pixels mean more edits. We observe a phenomenon where the ease of editing gradients was influenced by the information density of the data. On the PS-EMNIST dataset, we find that the gradients were more easily edited in regions with lower information

TABLE IV: Combination of ME with existing rehearsal methods on PS-CIFAR-100.

Method	Task-incremental PCL		Class-incremental PCL	
	A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$
AGEM [16]	27.379 $\pm$ 0.585	5.416 $\pm$ 0.851	3.374 $\pm$ 0.098	-10.462 $\pm$ 0.543
AGEM+ME	29.310 $\pm$ 0.309	7.060 $\pm$ 0.253	5.725 $\pm$ 0.122	-8.260 $\pm$ 0.075
ER [17]	47.324 $\pm$ 0.584	23.330 $\pm$ 0.762	9.317 $\pm$ 0.339	-5.337 $\pm$ 0.626
ER+ME	49.240 $\pm$ 0.149	24.480 $\pm$ 0.055	10.591 $\pm$ 0.376	-4.440 $\pm$ 0.139
GRASP [30]	45.710 $\pm$ 0.362	22.606 $\pm$ 0.415	8.027 $\pm$ 0.176	-2.465 $\pm$ 0.729
GRASP+ME	48.985 $\pm$ 0.143	24.015 $\pm$ 0.761	9.711 $\pm$ 0.048	-3.082 $\pm$ 0.253
SER [8]	46.121 $\pm$ 0.352	22.890 $\pm$ 0.690	8.184 $\pm$ 0.369	-2.827 $\pm$ 0.167
SER+ME	48.598 $\pm$ 0.125	24.367 $\pm$ 0.169	9.725 $\pm$ 0.124	-2.670 $\pm$ 0.154

density. This implies that the areas of the background are more susceptible to being edited. On the other hand, in the PS-CIFAR-100 and PS-ImageNet-TINY datasets, which contain complex images from various categories, we notice that gradients associated with higher information density pixels are more prone to being edited. These pixels contained more intricate features, textures, and patterns, making them more likely to be altered. These experimental results suggest that on simple datasets, where the task information is more explicit and the density of information is higher, rehearsal performs better. However, on more complex datasets, where there is greater uncertainty in the data, it is necessary to selectively edit the key elements of the original data to enhance its compatibility with future tasks. This targeted editing helps to improve the data's ability to support subsequent tasks by reducing uncertainty and enhancing relevant information.

t-SNE visualization of ME. To further examine whether the memory editing procedure introduces harmful distortions to the learned representations, we perform a t-SNE [67] visualization study under the PS-CIFAR-100 setting. Specifically, we analyze three types of embeddings extracted from the penultimate layer:

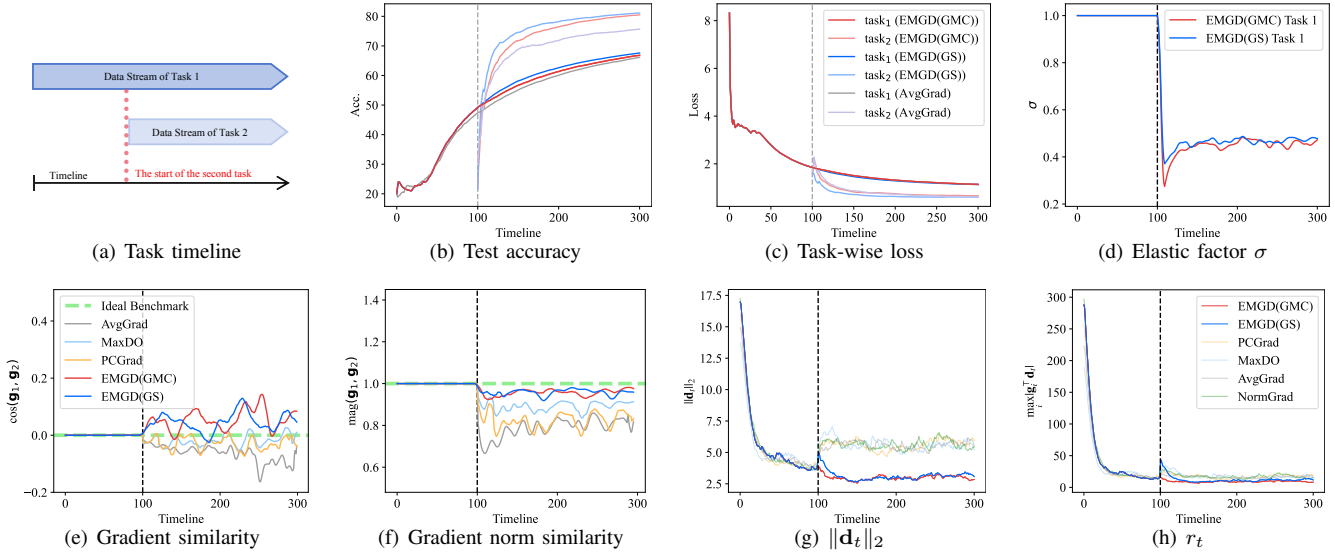


Fig. 5: Illustration of learning dynamics in a two-task PCL scenario on PS-CIFAR-10. (a) Task 2 is introduced one-third into the training of Task 1. (b) EMGD methods outperform AvgGrad in test accuracy. (c) Task 2 loss decreases more rapidly under elastic factor guidance. (d) Task weights σ initially prioritize the new task, then stabilize. (e) Cosine similarity between gradients indicates reduced conflict in EMGD. (f) Norm similarity $\text{mag}(\mathbf{g}_1, \mathbf{g}_2)$ shows balanced gradient magnitudes across tasks. (g) The update magnitude $\|\mathbf{d}(\theta_k)\|_2$ responds sharply at the task transition and then returns to a lower and more stable regime. (h) The residual $r_k = \max_i |\mathbf{g}_i^\top \mathbf{d}_t|$ provides an optimization-level view of the accumulation behavior of the EMGD update rule.

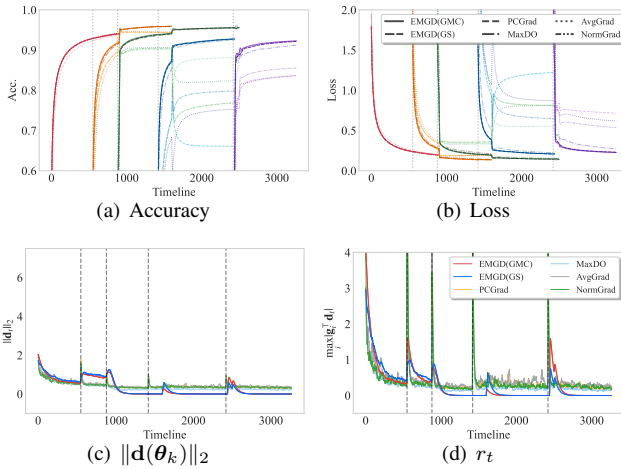


Fig. 6: Learning-stage visualization of optimization dynamics on PS-EMNIST. Dashed vertical lines indicate task arrivals.

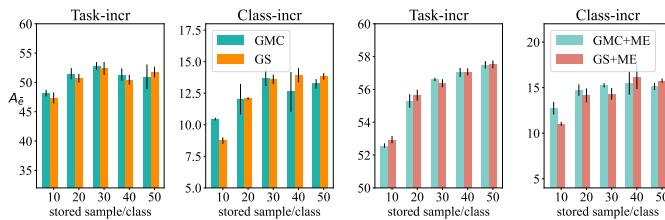


Fig. 7: Impact of memory size on PS-CIFAR-100.

(1) the current mini-batch samples before and after editing, (2) the rehearsal memory samples before and after editing, and (3) the test set samples after training with or without memory editing. As shown in Fig. 9, we observe that the editing operation leads to mild adjustments in the current batch samples, with only a small subset exhibiting visible shifts. More importantly, the rehearsal memory maintains a well-structured and separable layout even after editing,

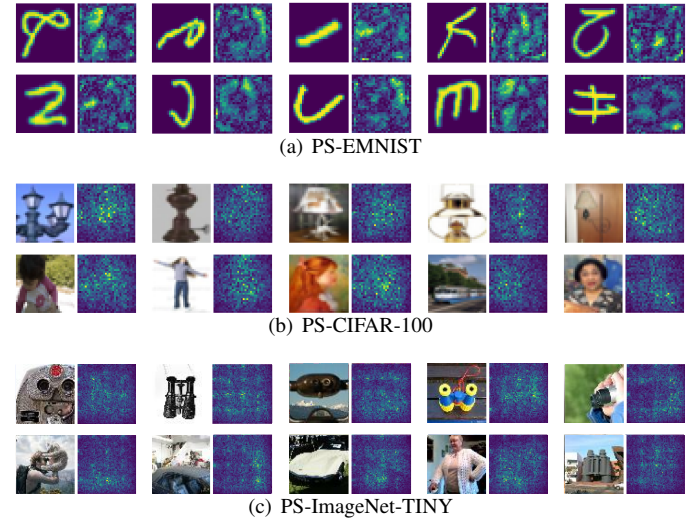


Fig. 8: Visualization of ME. Left: Stored images, Right: Normalized gradient.

indicating that the updates are local and do not corrupt the stored knowledge. For the test set, memory editing yields more compact and class-discriminative clusters, suggesting that the edited memory supports better generalization. These results collectively demonstrate that our editing procedure does not introduce destructive biases or distortions. Instead, it refines the memory content to enhance alignment, stability, and downstream performance in the PCL setting.

Generalization of ME to other rehearsal methods. To validate the generality of our proposed memory editing (ME) mechanism beyond the EMGD framework, we apply ME to several widely-used rehearsal-based continual learning methods, including AGEM [16], ER [17], GRASP [30], and SER [8]. These methods span gradient projection, naive replay, saliency-

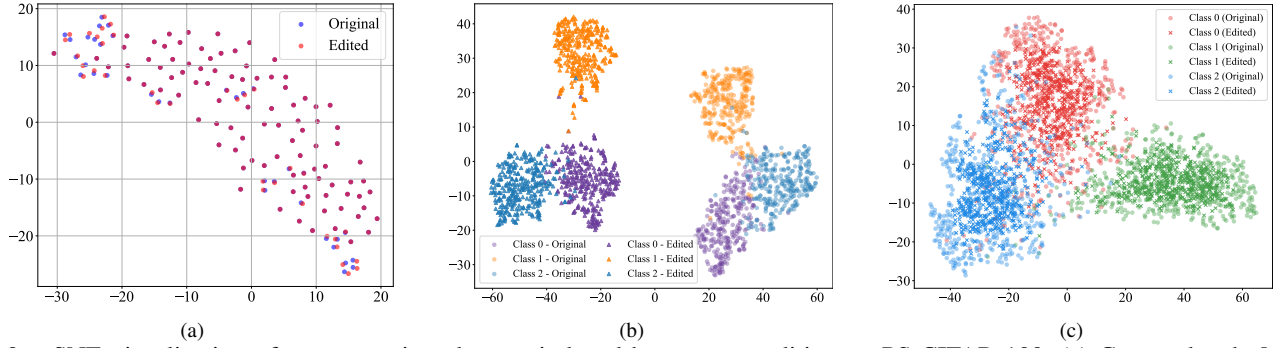


Fig. 9: t-SNE visualization of representation changes induced by memory editing on PS-CIFAR-100. (a) Current batch. Minor feature shifts indicate targeted adjustment. (b) Rehearsal memory. Edited features remain separable without destructive distortion. (c) Test set. The edited model shows clearer clustering and better class separation.

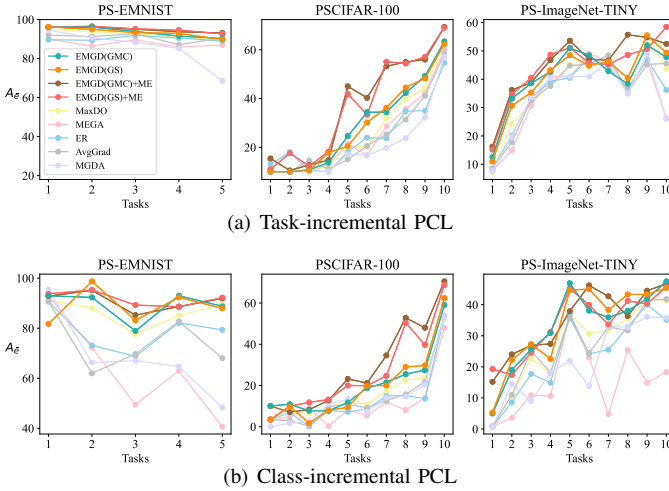


Fig. 10: Performance evolution across the PCL training process.

based selection, and memory filtering, offering a diverse evaluation basis. In our experiment, ME is integrated as a plug-in module: before rehearsal, each memory sample is refined by one-step gradient-guided editing (Eq. 12–13), without altering the underlying optimization logic or tuning new hyperparameters. The performance is evaluated on PS-CIFAR-100 under both task-incremental and class-incremental protocols. As shown in Table IV, ME consistently improves the average accuracy across all baselines in both settings. For instance, ER+ME achieves +1.9% and +1.3% gains in accuracy under task-incremental and class-incremental settings, respectively. Similarly, ME improves AGEM and GRASP by 1.9–2.3% across both metrics. These results suggest that ME is a general and effective enhancement strategy for boosting the utility of stored samples in memory-based continual learning.

E. Robustness Analysis of EMGD

PCL learning dynamics. As illustrated in Fig. 10, we track the performance evolution throughout the entire PCL training process. Each point i on the curve represents the time when task i has just completed training, and the corresponding value indicates the average test accuracy over all seen tasks, i.e., the tasks that have been trained up to that point. Across all three datasets and both PCL incremental settings, our method consistently yields superior accuracy compared to

TABLE V: All methods with ViT on PS-CIFAR-100.

Method	Type	Task-incremental PCL		Class-incremental PCL	
		A_e (%)	F_e (%)	A_e (%)	F_e (%)
MGDA [61]	MTL	50.157±0.224	-0.920±0.118	15.528±0.798	-0.335±0.169
PCGrad [75]	MTL	55.167±1.398	1.230±1.416	16.157±0.447	-0.863±0.238
CREST [34]	MTL	50.197±0.151	-0.310±0.200	16.012±0.153	-0.323±0.974
Aligned-MTL [62]	MTL	56.340±0.478	1.380±0.630	18.084±0.673	1.047 ±0.437
AGEM [16]	SCL	50.310±1.122	-0.880±0.628	14.938±0.247	0.462±0.053
ER [17]	SCL	52.320±3.075	-1.080±0.324	16.447±0.215	-1.702±0.810
LODE [38]	SCL	55.160±0.325	-0.960±0.145	17.039±0.712	1.017 ±0.238
MIND [9]	SCL	55.427±0.315	0.717±0.135	17.183±0.478	-1.734±0.744
MaxDO [50]	PCL	57.100±0.627	2.750 ±0.140	17.796±0.629	-1.813±0.898
EMGD(GMC)	PCL	61.203±0.223	0.197±0.851	23.388±0.105	-1.508±0.187
EMGD(GS)+ME	PCL	64.280 ±0.372	1.620±0.712	25.210 ±0.814	-1.433±0.582
EMGD(GS)	PCL	62.460±0.504	0.917±0.234	23.861±0.168	1.127 ±0.314
EMGD(GS)+ME	PCL	64.800 ±0.290	2.380 ±0.679	25.316 ±0.249	0.839 ±0.119

baseline approaches, demonstrating strong robustness and generalizability. Notably, EMGD maintains stable and high performance throughout the training timeline, highlighting its ability to effectively mitigate forgetting and optimize multi-task learning as new tasks arrive. These results affirm the effectiveness of our EMGD framework as a reliable solution for continual adaptation in PCL scenarios. Moreover, we compare two variants of elastic factor computation, namely GMC and GS. We observe that GMC excels in the early stages of training, while GS shows stronger performance in later stages. This can be attributed to the dynamics of task accumulation. In early stages, only a few tasks have been seen, so the learning process is more self-focused and benefits from GMC's independent weighting. In contrast, during later stages, where task interactions and conflicts are more prominent, GS effectively leverages gradient similarity to resolve inter-task interference and guide optimization in a more globally consistent manner.

Visualization of feature space. As shown in Fig. 11, we utilize t-SNE [67] to visualize the feature space and evaluate the effectiveness of our proposed memory editing strategy. The visualization results reveal that our method better preserves feature structure and inter-class uniformity throughout the PCL process. In contrast, the standard ER method suffers from semantic drift, resulting in fragmented and poorly clustered features. To quantify this observation, we compute the average cosine similarity between rehearsal samples and their corresponding class means. After training Task 1, both ER and our method exhibit high similarity. However, by the end of training all tasks, ER experiences a substantial drop in similarity, indicating that some rehearsal samples become out-

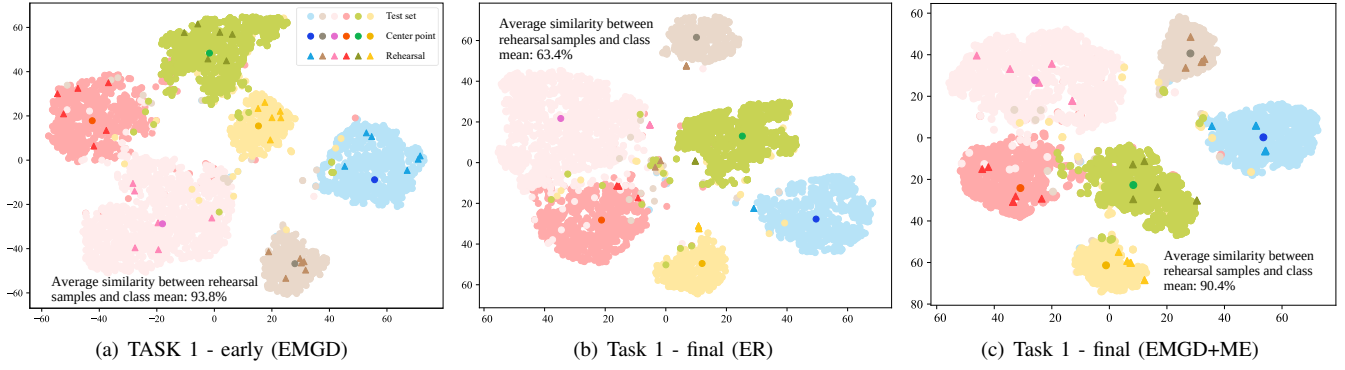


Fig. 11: t-SNE visualization on PS-EMNIST. We omit the “Task 1 - early (ER)” because it is quite similar to (a) TASK 1 - early (EMGD) at the early of PCL.

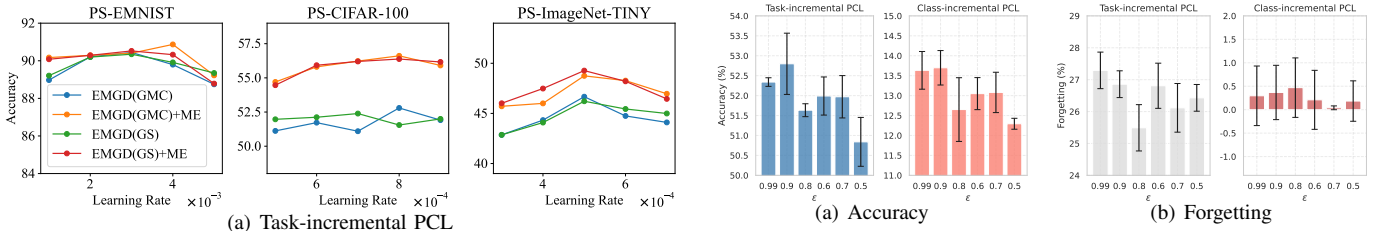


Fig. 14: Impact of the exponential momentum coefficient ϵ in GMC-based elastic factor computation.

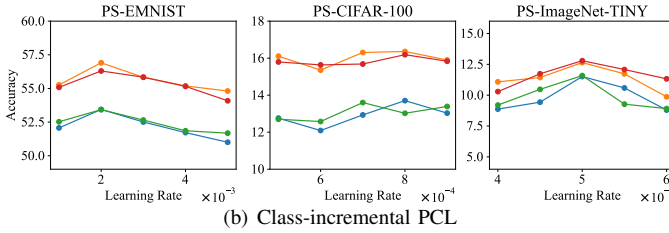


Fig. 12: Analysis on the learning rate.

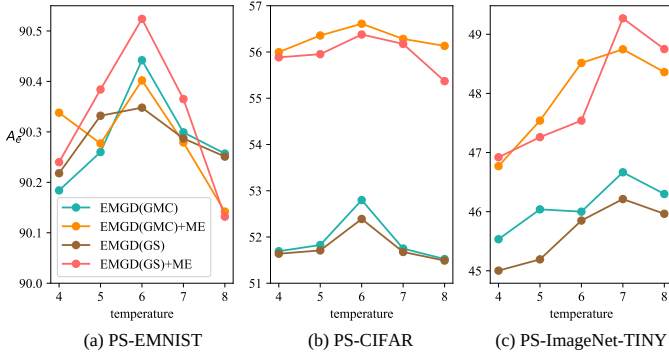


Fig. 13: Softmax temperature selection for three datasets.

of-distribution. In comparison, our memory editing approach maintains a consistently high similarity, suggesting enhanced representational stability and better resistance to forgetting.

EMGD with ViT backbone. In the context of PCL, the backbone model is task-agnostic and can be instantiated using any deep neural network. While different architectures are used in different datasets for implementation convenience, such as a 2-layer MLP for PS-EMNIST and ResNet-18 for PS-CIFAR-100 and PS-ImageNet-TINY, only one model is maintained and adapted across all tasks within each setting. As shown in Table V, we replace the ResNet-18 with Vision Transformers (ViT) on PS-CIFAR-100, and reevaluate the performance under task-incremental and class-incremental PCL settings.

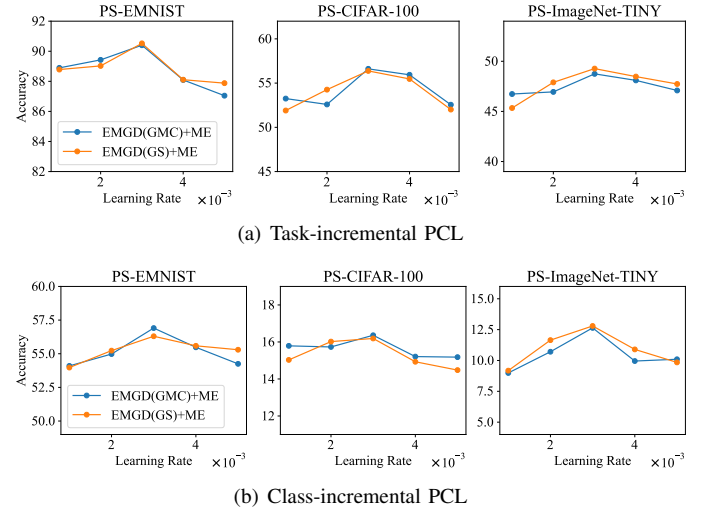


Fig. 15: Memory editing process learning rate.

The results show that our method can also be applied with ViT, further demonstrating its flexibility and model-agnostic nature.

Hyperparameter sensitivity and scalability. We assess the robustness of EMGD against key hyperparameters and its scalability *w.r.t.* model, memory, and data scales. Experiments are conducted on PS-EMNIST, PS-CIFAR-100, and PS-ImageNet-TINY under both task- and class-incremental settings. (1) *Learning rate.* Fig. 12 shows that EMGD maintains stable performance across a wide range of learning rates. Extremely small values (e.g., $1e-3$ in PS-EMNIST) slow convergence, while overly large rates (e.g., $> 5e-3$) can cause early instability, yet such instability is generally mitigated by elastic modulation. Both GMC and GS variants (w/ and w/o ME) show consistent trends, indicating that EMGD is not overly sensitive to this hyperparameter. (2) *Softmax temperature τ :*

TABLE VI: Performance comparison of EMGD under blurring PCL setting on CIFAR-100.

Method	Task increment		Class increment	
	A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$
AvgGrad	31.365 $\pm$ 0.767	18.070 $\pm$ 0.532	8.539 $\pm$ 0.253	-1.156 $\pm$ 0.169
MGDA [61]	31.492 $\pm$ 0.095	18.088 $\pm$ 0.337	11.220 $\pm$ 0.684	1.827 $\pm$ 0.749
PCGrad [75]	31.732 $\pm$ 1.636	18.276 $\pm$ 1.436	8.874 $\pm$ 0.510	-1.042 $\pm$ 0.490
AGEM [16]	18.502 $\pm$ 0.868	7.584 $\pm$ 0.244	9.477 $\pm$ 0.526	4.431 $\pm$ 0.293
ER [17]	31.044 $\pm$ 0.625	17.744 $\pm$ 0.056	8.142 $\pm$ 0.263	-1.511 $\pm$ 0.040
MEGA [29]	30.809 $\pm$ 0.336	18.074 $\pm$ 0.278	11.567 $\pm$ 0.414	4.051 $\pm$ 0.665
MaxDO [50]	32.391 $\pm$ 1.519	18.382 $\pm$ 1.305	10.912 $\pm$ 0.321	-0.271 $\pm$ 0.747
EMGD(GMC)	34.860 $\pm$ 0.975	20.244 $\pm$ 0.881	12.473 $\pm$ 0.115	1.785 $\pm$ 0.272
EMGD(GMC)+ME	45.139 $\pm$ 1.032	24.593 $\pm$ 1.428	18.227 $\pm$ 0.466	1.887 $\pm$ 0.264
EMGD(GS)	34.811 $\pm$ 1.685	20.212 $\pm$ 1.546	12.861 $\pm$ 0.904	2.089 $\pm$ 0.930
EMGD(GS)+ME	44.719 $\pm$ 0.840	24.321 $\pm$ 0.509	18.912 $\pm$ 0.793	2.742 $\pm$ 0.581

TABLE VII: Performance comparison of EMGD under multi-epoch PCL setting on CIFAR-100.

Method	Task increment		Class increment	
	A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$
AvgGrad	59.600 $\pm$ 0.437	-5.440 $\pm$ 0.271	19.390 $\pm$ 0.366	-17.990 $\pm$ 0.650
MGDA [61]	58.780 $\pm$ 6.844	-2.770 $\pm$ 6.250	21.057 $\pm$ 1.269	-14.886 $\pm$ 0.810
PCGrad [75]	56.720 $\pm$ 1.712	-6.210 $\pm$ 0.962	17.610 $\pm$ 0.105	-24.000 $\pm$ 1.422
AGEM [16]	49.949 $\pm$ 1.533	-14.084 $\pm$ 1.779	7.408 $\pm$ 0.411	-35.003 $\pm$ 0.896
ER [17]	59.322 $\pm$ 0.349	-6.774 $\pm$ 0.619	16.030 $\pm$ 0.787	-21.300 $\pm$ 0.058
MEGA [29]	60.710 $\pm$ 0.386	-2.779 $\pm$ 0.518	22.710 $\pm$ 0.317	-11.470 $\pm$ 0.731
MaxDO [50]	60.400 $\pm$ 1.862	-0.997 $\pm$ 2.326	22.940 $\pm$ 0.727	-19.020 $\pm$ 2.976
EMGD(GMC)	63.754 $\pm$ 1.080	-3.488 $\pm$ 0.862	26.984 $\pm$ 0.328	-11.944 $\pm$ 1.238
EMGD(GMC)+ME	65.222 $\pm$ 0.420	-2.586 $\pm$ 0.403	30.657 $\pm$ 0.263	-3.483 $\pm$ 0.707
EMGD(GS)	64.462 $\pm$ 0.524	-2.255 $\pm$ 0.734	25.885 $\pm$ 0.069	-8.741 $\pm$ 0.660
EMGD(GS)+ME	67.260 $\pm$ 0.926	-1.470 $\pm$ 0.143	29.445 $\pm$ 0.281	-2.607 $\pm$ 0.973

We examine the impact of τ in gradient weighting (Eq. (9), (10)) in Fig. 13. Lower τ values induce sharp, biased attention while higher values oversmooth task distinctions. A moderate τ yields optimal performance across datasets, highlighting its role in balancing task emphasis. (3) *Momentum coefficient in GMC* (ϵ , Eq. (9)). The momentum ϵ stabilizes gradient magnitude tracking in GMC. As shown in Fig. 14, $\epsilon = 0.9$ achieves the best trade-off between adaptability and stability. Larger values (e.g., 0.99) oversuppress updates while smaller ones (e.g., ≤ 0.7) amplify noise. A moderate ϵ ensures responsive yet reliable task modulation. (4) *Memory editing step size* η in Eq. (12). Fig. 15 shows that the pixel-level editing step size η influences performance significantly. Too small η results in ineffective edits, while too large values overmodify stored samples, harming generalization. Moderate η leads to better memory alignment and reduced task conflict.

F. EMGD in Related Settings

EMGD under blurring PCL. Blurring continual learning [6] differs from the conventional CL setting by allowing overlapping label spaces between tasks, which more accurately reflects real-world scenarios. To evaluate our method in this setting, we construct a blurring PCL benchmark based on CIFAR-100. Specifically, the 100 classes are divided into 9 overlapping tasks, each containing 20 classes, with a 50% label overlap between any two adjacent tasks. The results, summarized in Table VI, demonstrate that EMGD remains effective under this more realistic and challenging PCL scenario. Notably, for the two elastic factor strategies, we observe that GMC performs better in the task-incremental setting, whereas GS achieves superior results in the class-incremental setting. This is likely because, in class-incremental PCL where task IDs are

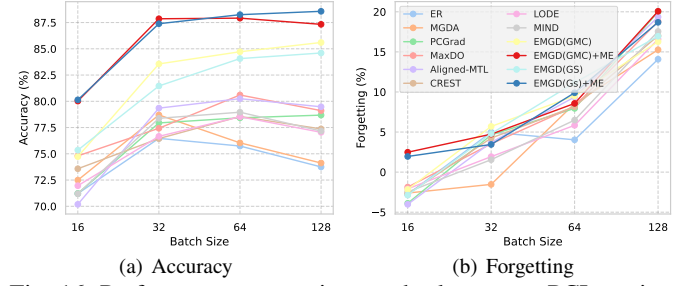


Fig. 16: Performance comparison under long-term PCL setting.

unavailable, the gradient similarity plays a more critical role in resolving cross-task interference. Furthermore, the results confirm the utility of our proposed memory editing mechanism, which enhances performance in rehearsal-based blurring PCL by improving compatibility across overlapping tasks.

EMGD under multi-epoch PCL. In our primary experiments, we adopt a single-epoch online training setting for each task. To further evaluate the robustness of our method, we conduct experiments using a multi-epoch setting, where each task is trained for 20 epochs while keeping the task sequence unchanged. The results, presented in Table VII, demonstrate that multi-epoch training leads to higher final accuracy compared to the single-epoch setting, as each task benefits from more thorough optimization and better convergence. However, we also observe that forgetting becomes more pronounced in this setting, due to the increased number of updates from subsequent tasks after earlier tasks have completed training. Despite this challenge, EMGD consistently outperforms all baselines in both accuracy and forgetting, and the performance gap even widens under the multi-epoch setting. These results validate the effectiveness and generalizability of our method in more realistic offline or long-horizon continual learning scenarios.

EMGD under Long-term PCL. To evaluate the effectiveness of our method under long-term training scenarios, we conduct experiments on PS-CIFAR-100 with 50 tasks while varying the batch size ($\{16, 32, 64, 128\}$). Smaller batch sizes naturally lead to a larger number of training iterations (from 173 to 1681 per task), thus simulating a more prolonged and potentially unstable learning process. We compare EMGD with and without memory editing against multiple baselines, including ER, MGDA, PCGrad, MaxDO, and MIND. As shown in Fig. 16, EMGD with memory editing consistently achieves higher accuracy and lower forgetting across all settings. The benefit becomes more pronounced in low-batch/high-iteration regimes, suggesting that memory editing helps the model preserve old task knowledge and reduce cross-task interference over time. These results validate the robustness and scalability of our method in long-term PCL.

EMGD in SCL. Although EMGD is designed for PCL, we further evaluate its applicability in the SCL setting, where tasks arrive sequentially and rehearsal buffers store samples from previous tasks. This naturally forms a two-task setup from the second task onward. Experiments are conducted on the standard non-overlapping benchmarks: Split-EMNIST, Split-CIFAR-100, and Split-Tiny-ImageNet, under both task- and class-incremental SCL protocols. Each sample is seen once, and performance is assessed using average accuracy (A_e) and

TABLE VIII: Comparisons on SCL dataset Split-CIFAR-10, Split-CIFAR-100 and Split-Tiny-ImageNet.

	Method	Venue	Split-EMNIST		Split-CIFAR-100		Split-ImageNet-TINY	
			A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$	A_e (%) $\uparrow$	F_e (%) $\uparrow$
Task-incremental SCL	AGEM [16]	ICLR'2019	76.733 $\pm$ 0.391	-5.595 $\pm$ 0.694	24.362 $\pm$ 0.973	6.237 $\pm$ 0.148	24.112 $\pm$ 0.178	-8.124 $\pm$ 0.293
	ER [17]	arXiv 2019	77.619 $\pm$ 0.458	-5.245 $\pm$ 0.787	49.396 $\pm$ 0.165	24.490 $\pm$ 0.263	29.411 $\pm$ 0.132	-3.183 $\pm$ 0.928
	MEGA [29]	NeurIPS'2020	78.119 $\pm$ 0.617	-5.461 $\pm$ 0.808	46.048 $\pm$ 0.573	22.925 $\pm$ 0.712	36.113 $\pm$ 0.543	-2.906 $\pm$ 0.271
	DER++ [11]	NeurIPS'2020	75.136 $\pm$ 0.756	-3.255 $\pm$ 0.271	45.989 $\pm$ 0.087	23.713 $\pm$ 0.376	36.332 $\pm$ 0.143	-1.141 $\pm$ 0.221
	CBRS [20]	ICML'2020	73.035 $\pm$ 0.466	-7.085 $\pm$ 0.197	43.289 $\pm$ 0.974	21.621 $\pm$ 0.684	28.455 $\pm$ 0.265	-6.249 $\pm$ 0.216
	CoPE [22]	ICCV'2021	80.179 $\pm$ 0.663	-3.723 $\pm$ 0.828	51.633 $\pm$ 0.402	22.873 $\pm$ 0.554	37.049 $\pm$ 0.451	1.269 $\pm$ 0.720
	AOP [28]	AAAI'2022	79.585 $\pm$ 0.243	-3.487 $\pm$ 0.128	49.584 $\pm$ 0.298	22.817 $\pm$ 0.472	36.877 $\pm$ 0.278	0.893 $\pm$ 0.477
	LODE [38]	NeurIPS'2023	71.035 $\pm$ 0.913	-8.510 $\pm$ 0.396	48.632 $\pm$ 0.174	21.076 $\pm$ 0.295	24.615 $\pm$ 0.198	-8.184 $\pm$ 0.422
	OnPro [69]	ICCV'2023	78.189 $\pm$ 0.375	-4.223 $\pm$ 0.182	47.743 $\pm$ 0.383	21.701 $\pm$ 0.682	35.967 $\pm$ 0.221	0.578 $\pm$ 0.287
	MIND [9]	AAAI'2024	81.375 $\pm$ 0.978	-4.981 $\pm$ 0.339	50.100 $\pm$ 0.443	23.580 $\pm$ 0.900	37.013 $\pm$ 0.811	1.343 $\pm$ 0.572
	NsCE [71]	NeurIPS'2024	79.064 $\pm$ 0.551	-5.698 $\pm$ 0.815	48.461 $\pm$ 0.563	21.694 $\pm$ 0.735	35.206 $\pm$ 0.696	-1.729 $\pm$ 0.319
	RanDumb [57]	arXiv'2024	80.027 $\pm$ 0.855	-3.249 $\pm$ 0.925	50.336 $\pm$ 0.138	22.189 $\pm$ 0.744	36.279 $\pm$ 0.443	0.928 $\pm$ 0.510
	MaxDO [50]	ICCV 2023	80.116 $\pm$ 0.470	-4.042 $\pm$ 0.776	52.860 $\pm$ 0.293	20.860 $\pm$ 0.461	38.140 $\pm$ 0.472	0.381 $\pm$ 0.819
	EMGD(GMC)	Ours	85.327 $\pm$ 0.811	-3.154 $\pm$ 0.213	54.030 $\pm$ 0.379	21.510 $\pm$ 0.137	40.463 $\pm$ 0.178	4.210 $\pm$ 0.021
	EMGD(GS)	Ours	84.437 $\pm$ 0.138	-3.942 $\pm$ 0.314	55.450 $\pm$ 0.497	23.350 $\pm$ 0.332	42.072 $\pm$ 0.313	3.204 $\pm$ 0.736
Class-incremental SCL	AGEM [16]	ICLR'2019	19.667 $\pm$ 0.687	-74.798 $\pm$ 0.697	2.143 $\pm$ 0.693	-14.783 $\pm$ 0.523	2.396 $\pm$ 0.102	-22.021 $\pm$ 0.128
	ER [17]	arXiv 2019	37.627 $\pm$ 0.207	-56.708 $\pm$ 0.218	13.570 $\pm$ 0.619	-17.043 $\pm$ 0.338	8.781 $\pm$ 0.135	-10.764 $\pm$ 0.114
	MEGA [29]	NeurIPS'2020	49.165 $\pm$ 0.616	-11.265 $\pm$ 0.711	13.760 $\pm$ 0.174	-9.690 $\pm$ 0.394	7.557 $\pm$ 0.239	-8.676 $\pm$ 0.240
	DER++ [11]	NeurIPS'2020	48.539 $\pm$ 0.375	-12.374 $\pm$ 0.467	12.620 $\pm$ 0.573	-16.010 $\pm$ 0.604	8.806 $\pm$ 0.522	-6.499 $\pm$ 0.541
	CBRS [20]	ICML'2020	35.805 $\pm$ 0.663	-58.122 $\pm$ 0.329	11.773 $\pm$ 0.430	-16.124 $\pm$ 0.269	6.462 $\pm$ 0.833	-8.109 $\pm$ 0.497
	CoPE [22]	ICCV'2021	46.220 $\pm$ 0.384	-21.042 $\pm$ 0.722	14.770 $\pm$ 0.432	-9.060 $\pm$ 0.259	11.164 $\pm$ 0.662	-5.589 $\pm$ 0.122
	AOP [28]	AAAI'2022	39.589 $\pm$ 0.488	-54.729 $\pm$ 0.610	14.350 $\pm$ 0.129	-15.450 $\pm$ 0.317	9.306 $\pm$ 0.403	-6.571 $\pm$ 0.178
	LODE [38]	NeurIPS'2023	32.072 $\pm$ 0.118	-61.043 $\pm$ 0.370	11.145 $\pm$ 0.372	-12.238 $\pm$ 0.138	7.135 $\pm$ 0.838	-9.462 $\pm$ 0.147
	OnPro [69]	ICCV'2023	44.809 $\pm$ 0.822	-23.728 $\pm$ 0.087	13.853 $\pm$ 0.648	-10.832 $\pm$ 0.176	8.337 $\pm$ 0.407	-6.562 $\pm$ 0.450
	MIND [9]	AAAI 2024	37.409 $\pm$ 0.618	-56.967 $\pm$ 0.936	14.963 $\pm$ 0.070	-8.130 $\pm$ 0.779	11.373 $\pm$ 0.129	-7.286 $\pm$ 0.158
	NsCE [71]	NeurIPS'2024	45.412 $\pm$ 0.376	-22.441 $\pm$ 0.273	13.245 $\pm$ 0.186	-12.595 $\pm$ 0.846	7.488 $\pm$ 0.194	-5.415 $\pm$ 0.813
	RanDumb [57]	arXiv'2024	46.830 $\pm$ 0.658	-22.316 $\pm$ 0.231	13.792 $\pm$ 0.152	-8.122 $\pm$ 0.012	9.536 $\pm$ 0.242	-5.679 $\pm$ 0.499
	MaxDO [50]	ICCV 2023	48.561 $\pm$ 0.561	-20.695 $\pm$ 0.459	15.290 $\pm$ 0.907	-6.330 $\pm$ 0.686	12.286 $\pm$ 0.158	-4.347 $\pm$ 0.261
	EMGD(GMC)	Ours	51.840 $\pm$ 0.319	-16.779 $\pm$ 0.645	16.390 $\pm$ 0.606	-8.510 $\pm$ 0.149	12.622 $\pm$ 0.705	-4.316 $\pm$ 0.184
	EMGD(GS)	Ours	50.902 $\pm$ 0.407	-15.573 $\pm$ 0.153	16.860 $\pm$ 0.251	-7.357 $\pm$ 0.420	12.785 $\pm$ 0.390	-3.534 $\pm$ 0.216

forward transfer (F_e). We compare EMGD against 13 SOTA rehearsal-based SCL methods, including AGEM [16], ER [17], MEGA [29], DER++[11], CBRS[20], CoPE [22], AOP [28], LODE [38], OnPro [69], MIND [9], NsCE [71], RanDumb [57], and MaxDO [50]. Results are shown in Table VIII. Across all datasets, EMGD achieves strong performance in both A_e and F_e . In the task-incremental setting, EMGD(GMC) achieves the best results on Split-EMNIST, while EMGD(GS) performs best on Split-CIFAR-100 and Split-Tiny-ImageNet, demonstrating robustness to varying task granularity. Notably, EMGD(GS) achieves up to +2.6% gain in A_e on Split-CIFAR-100 and +4.0% on Split-Tiny-ImageNet over the previous best (MaxDO). Under the more challenging class-incremental setting, EMGD again outperforms all baselines. EMGD(GS) excels on CIFAR-100 and Tiny-ImageNet, while EMGD(GMC) leads on EMNIST, suggesting GS is more effective when task identity is unavailable and inter-task alignment is critical. These results confirm EMGD's generalizability beyond PCL: its ability to modulate gradient-based memory integration enables robust learning in both parallel and serial settings. This highlights the potential of our unified framework to bridge diverse continual learning paradigms through principled gradient optimization and memory editing.

EMGD in MTL. We further evaluate EMGD in standard MTL settings, where all tasks are jointly optimized with shared parameters. Experiments are conducted on three overlapping datasets: Split-EMNIST, Split-CIFAR-100, and Split-Tiny-ImageNet. We compare EMGD against representative MTL methods, including MGDA [23], DWA [45], PCGrad [75], Nash-MTL [56], Aligned-MTL [62], and MaxDO [50]. As shown in Table IX, EMGD consistently outperforms all baselines across datasets. On Split-EMNIST, EMGD(GMC)

TABLE IX: MTL Comparisons on Split-CIFAR-10, Split-CIFAR-100, and Split-Tiny-ImageNet.

Method	Split-EMNIST	Split-CIFAR-100	Split-ImageNet-TINY
	A_e (%)	A_e (%)	A_e (%)
AvgGrad	85.137 $\pm$ 0.910	14.499 $\pm$ 0.542	18.106 $\pm$ 0.870
MGDA [23]	87.347 $\pm$ 0.494	15.018 $\pm$ 0.416	19.419 $\pm$ 0.079
DWA [45]	85.304 $\pm$ 0.412	13.601 $\pm$ 0.483	17.941 $\pm$ 0.548
PCGrad [75]	90.371 $\pm$ 0.389	17.110 $\pm$ 0.281	22.018 $\pm$ 0.178
Nash-MTL [56]	90.115 $\pm$ 0.097	16.334 $\pm$ 0.799	20.237 $\pm$ 0.869
Aligned-MTL [62]	91.437 $\pm$ 0.012	16.104 $\pm$ 0.721	21.194 $\pm$ 0.871
MaxDO [50]	92.984 $\pm$ 0.812	17.687 $\pm$ 0.827	21.134 $\pm$ 0.197
CREST [34]	92.108 $\pm$ 0.398	15.586 $\pm$ 0.821	19.283 $\pm$ 0.547
EMGD(GMC)	94.142 $\pm$ 0.167	18.490 $\pm$ 0.447	24.714 $\pm$ 0.197
EMGD(GS)	93.523 $\pm$ 0.318	19.218 $\pm$ 0.633	25.321 $\pm$ 0.713

achieves the highest accuracy of 94.14%, outperforming MaxDO and Aligned-MTL. On Split-CIFAR-100, EMGD(GS) reaches 19.22%, yielding a +1.53% gain over MaxDO. On Split-Tiny-ImageNet, EMGD(GS) again leads with 25.32%, surpassing EMGD(GMC) by +0.61% and outperforming PC-Grad by over +3.3%. These results demonstrate that EMGD remains effective even when task gradients are co-optimized. Its elastic modulation (based on GMC and GS) adapts to subtle inter-task dynamics and yields more balanced updates. Overall, EMGD proves to be a versatile optimization strategy applicable beyond continual learning, delivering robust performance in diverse multi-task settings with varying gradient structures.

G. Computational Efficiency Analysis

We evaluate EMGD's efficiency from three perspectives: memory consumption, per-iteration training time, and inference scalability under varying task concurrency. Experiments are conducted on PS-CIFAR-100 (task-incremental setting) using a single NVIDIA RTX 3090 GPU (24GB), and compared against

TABLE X: Comparisons of memory usage and training time for the proposed method and baselines under the PCL setting. Inference time with varying numbers of parallel tasks.

Method	Training (per iter)		Inference (time(s))			
	Memory(MB)	Time(s)	2-task	3-task	4-task	5-task
AvgGrad	3084.98	152.000	1.513	1.595	1.634	1.877
MGDA [61]	3244.71	159.463	1.539	1.736	1.798	1.983
PCGrad [32]	3272.48	164.900	1.657	1.629	1.935	2.204
MEGA [29]	3581.72	152.417	1.529	2.037	2.654	2.865
ER [17]	3084.98	149.902	1.568	1.676	1.877	1.974
MaxDO [50]	4424.80	208.266	1.763	1.813	2.305	2.463
EMGD(GMC)	3303.33	218.071	1.895	1.954	2.376	2.381
EMGD(GMC)+ME	4638.71	298.902	2.145	2.294	2.389	2.495
EMGD(GS)	3327.78	227.912	2.040	2.315	2.527	2.553
EMGD(GS)+ME	4709.32	311.732	2.326	2.717	2.723	2.742

representative baselines: AvgGrad, MGDA [61], PCGrad [32], MEGA [29], ER [17], and MaxDO [50]. As shown in Table X, EMGD introduces moderate training overhead due to elastic factor computation and optional memory editing. Among variants, EMGD(GS)+ME has the highest training cost (311.7s/iter, 4709MB), while EMGD(GMC) without memory editing remains lightweight (218.1s/iter, 3303MB), comparable to MaxDO (208.3s/iter). This supports the computational efficiency and scalability of EMGD. We further assess inference latency under varying task concurrency (2–5 parallel streams). EMGD variants exhibit near-linear scaling and remain competitive. For instance, at 5 concurrent tasks, EMGD(GS)+ME incurs only a minor latency increase compared to MaxDO (2.742s vs. 2.463s), supporting its suitability for latency-sensitive deployments. Overall, EMGD achieves a strong trade-off between performance and efficiency. Its modular design allows the elastic factor computation and memory editing to be toggled independently, enabling flexible adaptation to different resource constraints.

VIII. CONCLUSION AND FUTURE WORK

In this paper, we studied a new and more practical continual learning paradigm called Parallel Continual Learning (PCL), which enables the concurrent training of new and existing tasks, thereby bridging the gap between multi-task learning (MTL) and serial continual learning (SCL). To address the dual challenges of task conflict and catastrophic forgetting in PCL, we formulated the training process as a dynamic multi-objective optimization (MOO) problem. Based on this formulation, we introduced a novel optimization method called Elastic Multi-Gradient Descent (EMGD), which assigns adaptive elastic factors to each task to guide updates toward a Pareto-optimal direction. This ensures that task updates are balanced and non-destructive across the evolving task set. In addition, we proposed a gradient-guided memory editing mechanism, which improves the quality of rehearsal data by aligning it with the desired descent direction, thus further reducing inter-task interference and memory-induced forgetting. Extensive experiments across three PCL benchmark datasets demonstrate that EMGD consistently outperforms previous methods, including our prior work MaxDO and several state-of-the-art approaches in both MTL and SCL domains. These results validate the robustness and generalizability of EMGD across a range of continual learning scenarios.

Our current framework has several limitations. First, it is developed and evaluated in a centralized single-machine setting, and its extension to distributed, decentralized, or asynchronous PCL remains unexplored. Second, EMGD introduces additional optimization overhead due to multi-task gradient computation, constrained gradient aggregation, and memory editing, and its scalability under a much larger number of simultaneously active tasks still requires further study. Third, although the proposed elastic factors based on gradient magnitude change and gradient similarity are empirically effective, a more principled theoretical characterization of their design remains an open direction for future work. Fourth, our current evaluation mainly focuses on controlled classification-based benchmarks, and the learning-stage analysis is mostly conducted in relatively small-scale settings for interpretability. Extending EMGD to more complex real-world task streams and broader task forms remains an important direction for future work.

ACKNOWLEDGMENT

This work was jointly supported by the National Science and Technology Major Project (2022ZD0117901), the National Key R&D Program of China under Grants (No. 2023YFF0906200), the Beijing Natural Science Foundation (Grant No. L252033), and the National Natural Science Foundation of China (Nos. 62406323, 62572349, 62276182, 62506146, 62373355, 62306311, 62236010).

REFERENCES

- [1] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars. Memory aware synapses: Learning what (not) to forget. In *Proceedings of the European Conference on Computer Vision*, pages 139–154, 2018.
- [2] R. K. Ando, T. Zhang, and P. Bartlett. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(11):1817–1853, 2005.
- [3] A. Argyriou, T. Evgeniou, and M. Pontil. Convex multi-task feature learning. *Machine learning*, 73(3):243–272, 2008.
- [4] C. Atkinson, B. McCane, L. Szymanski, and A. Robins. Pseudo-rehearsal: Achieving deep reinforcement learning without catastrophic forgetting. *Neurocomputing*, 428:291–307, 2021.
- [5] B. Babcock, S. Babu, M. Datar, R. Motwani, and J. Widom. Models and issues in data stream systems. In *Proceedings of the ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, 2002.
- [6] J. Bang, H. Kim, Y. Yoo, J.-W. Ha, and J. Choi. Rainbow memory: Continual learning with a memory of diverse samples. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8218–8227, 2021.
- [7] A. Barzilay and K. Crammer. Convex multi-task learning by clustering. In *Artificial Intelligence and Statistics*, pages 65–73, 2015.
- [8] G. Bellitto, F. Proietto Salanitri, M. Pennisi, M. Boschini, L. Bonicelli, A. Porrello, S. Calderara, S. Palazzo, and C. Spampinato. Saliency-driven experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 37, pages 103356–103383, 2024.
- [9] J. Bonato, F. Pelosin, L. Sabetta, and A. Nicolosi. Mind: Multi-task incremental network distillation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 11105–11113, 2024.
- [10] E. V. Bonilla, K. Chai, and C. Williams. Multi-task gaussian process prediction. In *Advances in Neural Information Processing Systems*, 2007.
- [11] P. Buzzega, M. Boschini, A. Porrello, D. Abati, and S. Calderara. Dark experience for general continual learning: a strong, simple baseline. In *Advances in Neural Information Processing Systems*, volume 33, pages 15920–15930, 2020.
- [12] Y. Censor. Pareto optimality in multiobjective problems. *Applied Mathematics and Optimization*, 4(1):41–59, 1977.
- [13] K. Chai. Generalization errors and learning curves for regression with multi-task gaussian processes. In *Advances in Neural Information Processing Systems*, 2009.

- [14] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr. Riemannian walk for incremental learning: Understanding forgetting and intransigence. In *Proceedings of the European Conference on Computer Vision*, pages 532–547, 2018.
- [15] A. Chaudhry, A. Gordo, P. Dokania, P. Torr, and D. Lopez-Paz. Using hindsight to anchor past knowledge in continual learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6993–7001, 2021.
- [16] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny. Efficient lifelong learning with a-gem. In *International Conference on Learning Representations*, 2019.
- [17] A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. Torr, and M. Ranzato. On tiny episodic memories in continual learning. *arXiv preprint arXiv:1902.10486*, 2019.
- [18] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the International Conference on Machine Learning*, 2018.
- [19] Z. Chen, J. Ngiam, Y. Huang, T. Luong, H. Kretschmar, Y. Chai, and D. Anguelov. Just pick a sign: Optimizing deep multitask models with gradient sign dropout. In *Advances in Neural Information Processing Systems*, volume 33, pages 2039–2050, 2020.
- [20] A. Chrysakos and M.-F. Moens. Online continual learning from imbalanced data. In *International Conference on Machine Learning*, pages 1952–1961. PMLR, 2020.
- [21] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2021.
- [22] M. De Lange and T. Tuytelaars. Continual prototype evolution: Learning online from non-stationary data streams. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8250–8259, 2021.
- [23] J.-A. Désidéri. Multiple-gradient descent algorithm (mgda) for multiobjective optimization. *Comptes Rendus Mathématique*, 350(5):313–318, 2012.
- [24] A. Douillard, A. Ramé, G. Couairon, and M. Cord. Dytox: Transformers for continual learning with dynamic token expansion. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9285–9295, 2022.
- [25] J. Fliege and B. F. Svaiter. Steepest descent methods for multicriteria optimization. *Mathematical Methods of Operations Research*, 51:479–494, 2000.
- [26] R. M. French. Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135, 1999.
- [27] R. Groenendijk, S. Karaoglu, T. Gevers, and T. Mensink. Multi-loss weighting with coefficient of variations. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1469–1478, 2021.
- [28] Y. Guo, W. Hu, D. Zhao, and B. Liu. Adaptive orthogonal projection for batch and online continual learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 6783–6791, 2022.
- [29] Y. Guo, M. Liu, T. Yang, and T. Rosing. Improved schemes for episodic memory-based lifelong learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 1023–1035, 2020.
- [30] M. Y. Harun, J. Gallardo, J. Chen, and C. Kanan. Grasp: A rehearsal policy for efficient online continual learning. *arXiv preprint arXiv:2308.13646*, 2023.
- [31] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.
- [32] G. Hu, W. Zhang, H. Ding, and W. Zhu. Gradient episodic memory with a soft constraint for continual learning. *arXiv preprint arXiv:2011.07801*, 2020.
- [33] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [34] T.-T. Le, M. Nguyen, T. T. Nguyen, L. N. Van, and T. H. Nguyen. Continual relation extraction via sequential multi-task learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 18444–18452, 2024.
- [35] S. Lee, J. Zhu, and E. Xing. Adaptive multi-task lasso: with application to eqtl detection. In *Advances in Neural Information Processing Systems*, 2010.
- [36] D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.
- [37] Z. Li and D. Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, 2017.
- [38] Y.-S. Liang and W.-J. Li. Loss decoupling for task-agnostic continual learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 11151–11167, 2023.
- [39] W. Liao, H. Xiong, Q. Wang, Y. Mo, X. Li, Y. Liu, Z. Chen, S. Huang, and D. Dou. Muscle: Multi-task self-supervised continual learning to pre-train deep models for x-ray images of multiple body parts. In *Proceedings of Medical Image Computing and Computer Assisted Intervention*, pages 151–161, 2022.
- [40] X. Liao and L. Carin. Radial basis function network for multi-task learning. In *Advances in Neural Information Processing Systems*, pages 792–802, 2005.
- [41] B. Lin, F. Ye, and Y. Zhang. A closer look at loss weighting in multi-task learning. *arXiv preprint arXiv:2111.10603*, 2021.
- [42] X. Lin, H.-L. Zhen, Z. Li, Q.-F. Zhang, and S. Kwong. Pareto multi-task learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [43] J. Liu, S. Ji, and J. Ye. Multi-task feature learning via efficient l2, l1-norm minimization. In *Proceedings of the Conference on Uncertainty in Artificial Intelligence*, pages 339–348, 2009.
- [44] L. Liu, Y. Li, Z. Kuang, J. Xue, Y. Chen, W. Yang, Q. Liao, and W. Zhang. Towards impartial multi-task learning. In *International Conference on Learning Representations*, 2021.
- [45] S. Liu, E. Johns, and A. J. Davison. End-to-end multi-task learning with attention. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1871–1880, 2019.
- [46] W. Liu, T. Mei, Y. Zhang, C. Che, and J. Luo. Multi-task deep visual-semantic embedding for video thumbnail selection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3707–3715, 2015.
- [47] X. Liu, P. He, W. Chen, and J. Gao. Multi-task deep neural networks for natural language understanding. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pages 4487–4496, 2019.
- [48] X. Liu, C. Wu, M. Menta, L. Herranz, B. Raducanu, A. D. Bagdanov, S. Jui, and J. v. de Weijer. Generative feature replay for class-incremental learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshop*, pages 226–227, 2020.
- [49] D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [50] F. Lyu, Q. Sun, F. Shang, L. Wan, and W. Feng. Measuring asymmetric gradient discrepancy in parallel continual learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11411–11420, 2023.
- [51] F. Lyu, S. Wang, W. Feng, Z. Ye, F. Hu, and S. Wang. Multi-domain multi-task rehearsal for lifelong learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, number 10, pages 8819–8827, 2021.
- [52] J. Ma, Z. Zhao, X. Yi, J. Chen, L. Hong, and E. H. Chi. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 1930–1939, 2018.
- [53] A. Mallya, D. Davis, and S. Lazebnik. Piggyback: Adapting a single network to multiple tasks by learning to mask weights. In *Proceedings of the European Conference on Computer Vision*, pages 67–82, 2018.
- [54] A. Maurer, M. Pontil, and B. Romera-Paredes. Sparse coding for multitask and transfer learning. In *Proceedings of the International Conference on Machine Learning*, pages 343–351, 2013.
- [55] P. Moreno-Munoz, A. Artés-Rodríguez, and M. A. Alvarez. Continual multi-task gaussian processes. *arXiv preprint arXiv:1911.00002*, 2019.
- [56] A. Navon, A. Shamsian, I. Achituve, H. Maron, K. Kawaguchi, G. Chechik, and E. Fetaya. Multi-task learning as a bargaining game. In *Proceedings of the International Conference on Machine Learning*, pages 16428–16446, 2022.
- [57] A. Prabhu, S. Sinha, P. Kumaraguru, P. H. Torr, O. Sener, and P. K. Dokania. Randumb: A simple approach that questions the efficacy of continual representation learning. *arXiv preprint arXiv:2402.08823*, 2024.
- [58] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert. icarl: Incremental classifier and representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2001–2010, 2017.
- [59] L. Risheng, L. Yaohua, Z. Shangzhi, and J. Zhang. Gradient-based editing of memory examples for online task-free continual learning. In *Advances in Neural Information Processing Systems*, volume 34, pages 29193–29205, 2021.

- [60] A. Rosenfeld and J. K. Tsotsos. Incremental learning through deep adaptation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(3):651–663, 2018.
- [61] O. Sener and V. Koltun. Multi-task learning as multi-objective optimization. In *Advances in Neural Information Processing Systems*, 2018.
- [62] D. Senushkin, N. Patakin, A. Kuznetsov, and A. Konushin. Independent component alignment for multi-task learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20083–20093, 2023.
- [63] J. Serra, D. Suris, M. Miron, and A. Karatzoglou. Overcoming catastrophic forgetting with hard attention to the task. In *Proceedings of the International Conference on Machine Learning*, pages 4548–4557, 2018.
- [64] D. L. Silver, R. Poirier, and D. Currie. Inductive transfer with context-sensitive neural networks. *Machine Learning*, 73(3):313–336, 2008.
- [65] H. Tang and K. Jia. Discriminative adversarial domain adaptation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5940–5947, 2020.
- [66] S. Thrun and J. O’Sullivan. Discovering structure in multiple learning tasks: The tc algorithm. In *Proceedings of the International Conference on Machine Learning*, volume 96, pages 489–497, 1996.
- [67] L. Van der Maaten and G. Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- [68] J. Wang and J. Ye. Safe screening for multi-task feature learning with multiple data matrices. In *Proceedings of the International Conference on Machine Learning*, pages 1747–1756, 2015.
- [69] Y. Wei, J. Ye, Z. Huang, J. Zhang, and H. Shan. Online prototype learning for online continual learning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18764–18774, 2023.
- [70] C. Wu, L. Herranz, X. Liu, J. van de Weijer, B. Raducanu, et al. Memory replay gans: Learning to generate new categories without forgetting. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [71] W. Xinrui, C. Geng, W. Wan, S.-Y. Li, and S. Chen. Forgetting, ignorance or myopia: Revisiting key challenges in online continual learning. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics and the International Joint Conference on Natural Language Processing*, 2024.
- [72] Y. Yang and T. M. Hospedales. A unified perspective on multi-domain and multi-task learning. In *International Conference on Learning Representations*, 2014.
- [73] J. Yoon, E. Yang, J. Lee, and S. J. Hwang. Lifelong learning with dynamically expandable networks. In *International Conference on Learning Representations*, 2018.
- [74] K. Yu, V. Tresp, and A. Schwaighofer. Learning gaussian processes from multiple tasks. In *Proceedings of the International Conference on Machine Learning*, pages 1012–1019, 2005.
- [75] T. Yu, S. Kumar, A. Gupta, S. Levine, K. Hausman, and C. Finn. Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 33, pages 5824–5836, 2020.
- [76] F. Zenke, B. Poole, and S. Ganguli. Continual learning through synaptic intelligence. In *Proceedings of the International Conference on Machine Learning*, pages 3987–3995, 2017.
- [77] B. Zhang, A. Bapna, R. Sennrich, and O. Firat. Share or not? learning to schedule language-specific capacity for multilingual translation. In *International Conference on Learning Representations*, 2020.
- [78] J. Zhang, Z. Ghahramani, and Y. Yang. Learning multiple related tasks using latent independent component analysis. In *Advances in Neural Information Processing Systems*, volume 18, 2005.
- [79] W. Zhang, L. Deng, and D. Wu. Overcoming negative transfer: A survey. *arXiv preprint arXiv:2009.00909*, 2020.
- [80] Y. Zhang and Q. Yang. A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering*, 34(12):5586–5609, 2021.
- [81] Z. Zhao, L. Hong, L. Wei, J. Chen, A. Nath, S. Andrews, A. Kumthekar, M. Sathiamoorthy, X. Yi, and E. Chi. Recommending what video to watch next: a multitask ranking system. In *Proceedings of the ACM Conference on Recommender Systems*, pages 43–51, 2019.



Fan Lyu is currently a Postdoctoral Research Fellow at the Computer Vision Center (CVC), Universitat Autònoma de Barcelona (UAB). Before he joined CVC, he was a Postdoctoral Researcher at the New Laboratory of Pattern Recognition (NLPR), Institute of Automation, Chinese Academy of Sciences (CASIA). He obtained his Ph.D. from the College of Intelligence and Computing (CIC), Tianjin University (TJU), China. His research interests focus on sustainable artificial intelligence and open-world learning.



Wei Feng received the B.S. and M.Phil. degrees in computer science from Northwestern Polytechnical University, China, in 2000 and 2003, respectively, and the Ph.D. degree in computer science from the City University of Hong Kong in 2008. He is currently a Full Professor with CIC, TJU, China. His major research interests include active robotic vision and visual intelligence.



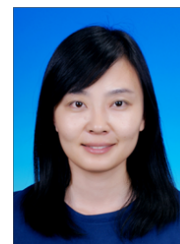
Yuepan Li is currently a master student with CIC, TJU, China. Her major research interest is continual learning.



Qing Sun received the M.Phil. degrees in CIC, TJU, China. She is currently a researcher at Tencent Inc. Her major research interest is continual learning.



Fanhua Shang received the PhD degree in circuits and systems from Xidian University, Xi’an, China, in 2012. He is a professor with the CIC, TJU, China. His current research interests include machine learning, data mining, pattern recognition, and computer vision.



Liang Wan received the B.Eng. and M.Eng. degrees in computer science and engineering from Northwestern Polytechnical University, China, in 2000 and 2003, respectively, and the Ph.D. degree in computer science and engineering from the Chinese University of Hong Kong in 2007. She is currently a Full Professor with the CIC, TJU, China. Her main research interests include image processing, image segmentation, and medical image analysis.



Liang Wang received the B.Eng. and M.Eng. from Anhui University, China, in 1997 and 2000, respectively, and the Ph.D. from CASIA, in 2004. From 2004 to 2010, he was a Research Assistant with Imperial College London, and Monash University, Australia, a Research Fellow with the University of Melbourne, Australia, and a Lecturer with the University of Bath. He is currently a Full Professor of the Hundred Talents Program with NLPR, CASIA. His major research interests include machine learning, pattern recognition, and computer vision.